

EBOOK · 1ª EDIÇÃO · 2026

Do Zero ao SaaS.

*Do zero ao primeiro SaaS
faturando em 30 dias —
sem saber programar.*

5 IAS · 7 FASES · 1 PRODUTO EM PRODUÇÃO

MÉTODO COMPLETO

DOZEROAOSAAS.COM.BR

Do Zero ao SaaS

Do zero ao primeiro SaaS faturando em 30 dias

Sem saber programar.

Autor: Equipe Do Zero ao SaaS **Edição:** 1ª edição — 2026 **Método:** 5 ferramentas de IA · 7 fases · 1 produto em produção

Aviso antes da primeira página

Este não é um curso de programação.

Este também não é um curso de "vibe coding" de guru do TikTok que te promete faturar milhões em 30 dias se você só "seguir o fluxo".

É um **método**. Uma sequência de decisões e ações, em ordem, para sair do ponto onde você tem uma ideia na cabeça e chegar no ponto onde existe um produto real rodando numa URL pública, aceitando pagamento no Stripe, com um primeiro cliente que pagou com dinheiro de verdade.

Se você quer entender por que React é melhor que Vue, este ebook não é pra você. Se você quer aprender a projetar sistemas distribuídos, este ebook não é pra você. Se você quer ficar "sabendo de tudo" sem nunca publicar nada, este ebook **especialmente** não é pra você.

Agora: se você quer parar de esperar, parar de aprender infinitamente, e fazer sua primeira coisa real acontecer usando IA como alavanca — estamos no lugar certo.

Não há magia aqui. Há método.

Vamos.

Como usar este ebook

- **Leia a Parte 1 antes de qualquer coisa.** Ela define como você vai pensar durante o método inteiro. Pular essa parte é como montar móvel sem ler o manual: vai dar certo, mas vai sangrar.
- **A Parte 2 apresenta suas cinco ferramentas.** Leia uma por dia se quiser decantar, ou de uma vez se estiver com pressa.
- **A Parte 3 é o motor operacional.** O ciclo dos 5 passos e dois estudos de caso reais. Releia antes de começar cada fase da trilha.
- **A Parte 4 é a joia da coroa** — os dois "pulos do gato" que economizam tempo, dinheiro e frustração. Releia toda semana nos primeiros 30 dias.
- **A Parte 5 fecha o ciclo** — troubleshooting e o que fazer depois que você publicou.

Você tem 3 companheiros além deste PDF:

1. **O Dashboard interativo** (URL no seu email) — onde ficam os prompts prontos, a biblioteca de 100+ prompts e a calculadora.
2. **Os 8 templates** na pasta do produto — PRDs, schemas, checklists.
3. **A comunidade** (link no seu email) — onde você faz perguntas quando travar.

Não leia o ebook de uma vez tentando absorver tudo. Leia a parte da trilha em que você está. Aplique. Volte. Leia a próxima.

O que você vai ter ao final dos 30 dias

Não vou te prometer milhão no primeiro mês. Vou te prometer o que eu sei que acontece se você seguir o método:

- Uma URL pública (seu domínio ou subdomínio Vercel) onde seu SaaS roda.
- Login funcionando (email/senha e Google).
- Stripe integrado em modo live, aceitando pagamento real.
- Uma landing de vendas no ar com copy que converte.
- Pelo menos uma venda registrada — mesmo que seja sua mãe comprando pra testar.
- Conhecimento pra repetir o processo quantas vezes quiser.

Isso não é pouco. Isso é mais do que 95% das pessoas que começam a "aprender programação" conseguem em 2 anos.

Boa leitura.

PARTE 1 — Por que esse método existe

Duas ideias pra decantar antes de tocar numa única ferramenta.

Capítulo 1 — O caos invisível

Você já reparou que, toda vez que você tenta construir alguma coisa com IA, abre pelo menos quatro abas no navegador?

Uma pro Claude, porque alguém falou que ele é bom em raciocínio. Outra pro ChatGPT, porque você já tinha conta. Uma pro Gemini, porque o Google empurrou. E uma pro Lovable, porque viralizou num reel semana passada.

Aí você pula de aba em aba, tentando lembrar em qual delas colou o último prompt, qual das versões do código funcionava, em qual chat estava o schema do banco que você pediu. Seis horas depois, você tem: muita coisa aberta, nenhuma linha pronta, e a sensação crescente de que talvez o problema seja você.

Não é você. É método.

Ou melhor: a falta dele.

Existem hoje centenas de tutoriais de 20 minutos no YouTube te mostrando "como fazer um SaaS em uma tarde com IA". Você assiste. Tenta reproduzir. Trava na terceira etapa porque a ferramenta do youtuber era outra versão, ou porque o erro que apareceu na sua tela não apareceu na dele, ou porque você esqueceu de configurar uma variável que ele passou voando.

O problema é estrutural. Você está tentando dirigir quatro carros ao mesmo tempo sem saber pra onde está indo.

O teste do espelho

Faça esse diagnóstico rápido. Marque quantos itens são verdade pra você hoje:

- Eu abro Claude/GPT/Gemini meio que aleatoriamente, conforme lembro de qual está aberto.
- Meus prompts começam com "oi, tudo bem?" ou equivalente.
- Eu pago Lovable e queimo os créditos em 3 dias sem ter nada publicado.
- Quando o código gera um erro, eu copio o erro no mesmo chat onde estava gerando o código, mesmo que o chat já tenha 40 mensagens.
- Eu nunca sei se devo pedir o código pronto ou pedir o plano primeiro.
- Eu tenho uma pasta "Ideias de SaaS" com 20 ideias, zero começadas.
- Toda vez que eu tento aprender, aparece uma ferramenta nova e eu paro pra testar ela.

Se você marcou 3 ou mais, bem-vindo. Você é o público desse método.

A boa notícia: o problema é solucionável e não é longo. O que você precisa não é mais conhecimento. É ordem.

Por que o "prompt mágico" não resolve

Existe uma fantasia, muito vendida em packs do Hotmart a R\$ 27, de que existe algum prompt específico, escrito de uma forma especial, que vai transformar uma IA genérica num programador sênior. O tal do "mega prompt".

Isso é mentira de quatro formas diferentes:

Primeiro, nenhum prompt isolado substitui a falta de clareza sobre o que você quer construir. Se você não sabe o que é o produto, nenhum prompt vai descobrir por você.

Segundo, um prompt bom resolve um problema. Construir um SaaS são dezenas de problemas encadeados. Um prompt não basta. Uma sequência de prompts, feita na ordem certa, para ferramentas diferentes, basta.

Terceiro, ferramentas de IA têm forças e fraquezas específicas. Um prompt genial pro Claude pode gerar lixo no GPT. Pedir imagem pro Claude é como pedir que ele desenhe: não vai sair bem. Pedir PRD pro GPT é como pedir poema pro Excel: vai sair, mas não vai ser bom.

Quarto, mesmo o melhor prompt do mundo esbarra num limite duro: você precisa saber **quando usar cada ferramenta** e **o que fazer com a resposta**. Isso é método, não prompt.

O que este ebook faz diferente

Vou te dar uma fórmula que tem três camadas:

1. **Um mapa mental** — quais cinco ferramentas existem e qual o papel de cada uma.

2. **Um fluxo operacional** — a ordem em que você chama cada uma, em cada fase do seu projeto.
3. **Prompts calibrados** — palavra por palavra, testados, pra cada papel de cada ferramenta.

As três camadas juntas é o que chamo de orquestra. Porque é isso que você vai aprender a fazer: orquestrar. Não tocar violino. Reger.

A diferença entre um músico e um maestro é que o músico precisa dominar um instrumento. O maestro precisa saber quando cada instrumento deve entrar, em que intensidade, por quanto tempo. O maestro não precisa tocar violino melhor que o violinista — precisa saber que aquela passagem pede violino, não trompete.

Você vai virar maestro.

Capítulo 2 — Mindset: você é o maestro, não o músico

Precisamos desfazer uma confusão comum antes de seguir: **aprender a programar** e **aprender a construir produtos com IA** são duas habilidades diferentes.

A primeira leva 2 a 5 anos, exige matemática discreta, estruturas de dados, paradigmas de programação, frameworks que mudam a cada 18 meses. É uma carreira.

A segunda pode ser aprendida em 30 dias e usada pro resto da vida. É um método.

As duas habilidades têm alguma sobreposição — você vai, sim, encostar em código em alguns momentos. Mas a diferença de atitude é gigante. O programador pensa em **como** fazer. Você vai pensar em **o quê** fazer, e delegar o "como" pra IA certa.

A analogia que resolve

Imagina que você decidiu construir uma casa. Existem dois caminhos:

Caminho A — virar pedreiro. Você aprende a misturar cimento, a assentar tijolo, a instalar fiação elétrica, a furar cerâmica sem trincar. Depois de 3 anos aprendendo, você constrói sua casa. É sua, ficou ótima. E você agora é pedreiro.

Caminho B — virar contratante/arquiteto. Você desenha a planta, define os cômodos, escolhe os materiais, contrata os pedreiros certos pras partes certas, supervisiona o trabalho, aprova os resultados. Em 3 meses a casa está pronta. É sua, ficou ótima. E você agora sabe gerenciar obras.

O caminho tradicional de "aprender a programar" é o A. O método Do Zero ao SaaS é o B.

Nos dois casos, a casa fica em pé. A diferença é que no B você paralelizou o trabalho com gente (ou IA) que faz cada parte melhor e mais rápido do que você jamais faria.

O que isso significa na prática

Se você aceita ser maestro em vez de músico, algumas coisas mudam:

Você para de tentar entender tudo. Não é seu trabalho saber por que React usa hooks em vez de classes. É seu trabalho saber que quando a IA gera um componente, ela vai usar hooks, e que isso é o padrão atual do mercado. Ponto. Você delega o "por quê" pra quem gerou.

Você começa a pensar em papéis. Claude é bom em X. Gemini é bom em Y. GPT é bom em Z. Lovable faz W. Manus executa tarefas longas. Sua tarefa como maestro é mapear cada pedaço do seu projeto pro especialista certo.

Você para de escrever prompts gigantes tentando resolver tudo de uma vez. Prompt grande demais é sintoma de que você não decidiu o que quer. Decide primeiro, prompta depois. Prompt pequeno e certo vence prompt longo e vago, sempre.

Você aceita que a IA erra. Às vezes a IA gera código que não funciona, resposta que não faz sentido, análise que tem número inventado. Isso é parte do jogo. Seu trabalho como maestro é ter critério pra detectar o erro e mandar de volta pra corrigir. Nenhum maestro aceita a primeira execução da orquestra sem ajustes.

O custo oculto de cada escolha errada

Uma das coisas que separa quem constrói de quem fica só estudando é a **consciência de custo**.

Todo prompt tem um custo. Todo minuto que você gasta esperando uma resposta tem um custo. Todo crédito queimado em iteração desnecessária tem um custo.

No método, você vai aprender coisas específicas sobre custo que mudam tudo:

- Por que iterar dentro do Lovable é a armadilha mais cara do mercado hoje.
- Como o Claude Code pode fazer em centavos o que o Lovable cobra em dezenas de dólares.
- Quando vale pagar Gemini Pro e quando a versão gratuita é suficiente.
- Por que "deixar o Manus rodando a noite toda" pode te dar um relatório de 80 páginas inútil.

Tudo isso está nas próximas partes. O que você precisa internalizar agora é: **dinheiro é feedback**. Se você está queimando 50 dólares por semana e não tem produto, a ferramenta não está errada — o uso está.

Mapa mental para fixar

Antes de seguir pro Capítulo 3, imprima ou anote num papel as três frases abaixo. Elas são sua bússola no método inteiro:

- 1. Eu não preciso saber como a ferramenta faz. Preciso saber quando usá-la.**
- 2. Prompt grande é sintoma de indecisão. Decido primeiro, prompt depois.**
- 3. Todo output de IA é rascunho até eu validar. Eu sou o maestro, não a plateia.**

Se essas três frases estiverem vivas na sua cabeça enquanto você lê a Parte 2, o resto do método se encaixa sozinho.

PARTE 2 — As cinco ferramentas e seus papéis

Você vai reger cinco instrumentos. Cada um com função específica. Saber o papel de cada um é 80% do método.

Capítulo 3 — Claude, o Arquiteto

Claude é a ferramenta que você usa quando precisa **pensar bem**. Não rapidamente — bem.

Se você pudesse resumir a personalidade do Claude em uma frase, seria: "aquele amigo engenheiro que responde devagar mas responde certo". Ele é construído pra contexto longo, raciocínio estruturado e saída técnica de qualidade. É o melhor da categoria em código, escrita técnica e análise que exige cuidado.

O que Claude faz melhor que os outros

Escrever PRDs e documentação técnica. Você descreve o projeto, Claude entrega um documento estruturado, coerente, com seções bem definidas. Ele não inventa bullet points aleatórios — ele raciocina sobre o que o PRD precisa cobrir e entrega o que falta.

Projetar arquitetura e schema de banco. Se você precisa decidir como as tabelas do seu Supabase vão se relacionar, Claude pensa em normalização, em índices, em segurança (RLS), e te entrega o schema como código SQL pronto pra colar. Mais importante: ele explica as decisões por baixo do schema, pra você aprender.

Auditar código gerado por outra IA. Quando o Lovable gera um código e você tem medo de subir em produção, Claude é quem audita. Ele enxerga bugs, brechas de segurança, queries ineficientes, e te mostra linha por linha o que precisa arrumar.

Resolver problema complexo em contexto longo. Quando você tem um problema que envolve vários arquivos, várias camadas, várias decisões interconectadas, Claude aguenta o contexto inteiro sem perder o fio. GPT ia confundir. Gemini ia alucinar. Claude mantém a linha.

Escrita técnica ou editorial de qualidade. Se você precisa de um texto que soe inteligente sem soar chato, Claude é a melhor opção. Para escrita de marketing puro (copy de venda), GPT ganha. Mas pra tudo que envolve técnica + texto, Claude é o padrão.

O que Claude NÃO faz bem

- **Brainstorm rápido e criativo.** Ele pensa demais antes de responder. Se você quer 50 ideias em 2 minutos, use GPT.
- **Gerar imagens.** Ele não faz. Use GPT (DALL-E) ou ferramenta específica.
- **Pesquisa web profunda com entregável pronto.** Ele pesquisa, mas entrega em formato conversa. Se você quer relatório formatado em PDF, use Manus.
- **Código de 300 linhas num único prompt sem contexto.** Ele tenta, mas vai alucinar em partes. Prefira sessões iterativas menores.

Como Claude espera ser tratado

Claude funciona melhor quando você:

1. **Dá contexto antes da tarefa.** Em vez de "escreva um PRD", use "Sou empreendedor solo, stack Lovable+Supabase, quero construir [X]. Escreva o PRD."
2. **Pede uma coisa de cada vez.** "Gere o PRD" → recebe → "agora o schema" → recebe → "agora critique tudo". É mais produtivo que "faça o PRD, o schema e critique tudo de uma vez".
3. **Usa Projects quando o assunto vai durar dias.** Projects permitem que o Claude mantenha contexto entre sessões. Perfeito pro projeto da sua trilha.
4. **Pede para ele discordar.** Se você fala "concorda?" ele tende a concordar. Se você fala "seja cético, aponte falhas no meu raciocínio", ele muda de postura.

Artifacts, Projects, Memory — o que importa

Três recursos do Claude que você vai usar bastante:

- **Artifacts:** quando Claude entrega código ou documento longo, ele abre um painel lateral que você pode copiar, editar, reutilizar. Perfeito pra PRDs e schemas.
- **Projects:** uma "pasta" dentro do Claude que mantém contexto entre conversas. Crie um Project chamado "SaaS Equipe Do Zero ao SaaS" e todas as conversas sobre seu produto acontecem lá. Ele lembra do que você falou sem você precisar repetir.
- **Memory:** ele lembra de fatos seus entre conversas (seu nome, seu projeto, suas preferências). Você pode pedir explicitamente pra ele lembrar de algo: "lembre que meu stack é Lovable + Supabase".

Prompts-chave pra Claude (use estes literalmente)

Pra validar uma ideia:

Aja como um investidor cético. Vou descrever uma ideia de SaaS.
Sua tarefa: listar os 5 maiores riscos em ordem de gravidade, identificar 3 suposições que estou assumindo como verdade, e sugerir o menor experimento possível para validar a suposição mais crítica. Se a ideia for fraca, fale sem rodeios.

IDEIA: [descreva em 2-3 frases]

PÚBLICO: [quem usaria]

MODELO DE RECEITA: [como ganha dinheiro]

Pra escrever um PRD:

Atue como Product Manager experiente. Gere um PRD completo para:

PROJETO: [nome + 1 parágrafo]

PROBLEMA: [a dor que resolve]

USUÁRIO-ALVO: [persona]

STACK: Lovable (React/TS) + Supabase + Stripe

Estrutura:

1. Visão e objetivo
2. Problema e contexto
3. Personas detalhadas
4. Proposta de valor
5. Escopo do MVP (o que ENTRA e o que FICA DE FORA)
6. Fluxos de usuário principais
7. Schema de banco (tabelas e relações em SQL)
8. Autenticação e segurança (RLS)
9. Integrações externas
10. Métricas de sucesso
11. Riscos e mitigações

Seja específico. Liste em vez de parafrasear. Onde houver decisão, recomende UMA opção com 1 frase de justificativa.

Pra auditar código:

Vou colar um trecho de código gerado por outra IA. Faça auditoria em 4 níveis:

1. BUGS: o que vai quebrar
2. SEGURANÇA: RLS ausente, credenciais expostas, validação
3. PERFORMANCE: queries N+1, re-renders, missing indexes
4. MANUTENIBILIDADE: código duplicado, nomes ruins

Pra cada achado: gravidade (crítico/médio/baixo), linha específica, correção em código. Ordene do mais grave ao menor. Nada de elogiar.

[cole o código]

Pro ciclo da trilha inteira, crie um Project. Chame de "Do Zero ao SaaS — [seu projeto]". Coloque lá: o PRD, o schema, as decisões-chave. Toda conversa nova dentro do Project já entra com esse contexto.

Capítulo 4 — Gemini, o Tutor e Leitor

Gemini é a ferramenta que você abre quando **não sabe como fazer alguma coisa**.

Se Claude é o engenheiro que planeja, Gemini é o professor que te guia. Ele é multimodal, tem contexto gigantesco, e tem um tom didático que as outras IAs não conseguem. Quando você manda um screenshot do painel do Supabase perguntando "o que cliço agora?", Gemini olha a imagem, entende o que aparece, e te diz qual botão apertar.

Essa habilidade específica — **ler print e explicar** — faz dele insubstituível no fluxo de um iniciante.

O que Gemini faz melhor que os outros

Ler screenshots e explicar. Você tira print do painel do Stripe, do erro do console, do painel do Supabase, do DNS da Cloudflare. Gemini olha, entende, e te explica o que cada campo significa. Claude também lê imagem, mas é menos paciente em "me ensine". Gemini tem a vocação de tutor.

Aguentar contexto gigantesco. Gemini tem a maior janela de contexto do mercado (milhões de tokens). Isso significa que você pode mandar a documentação INTEIRA de uma ferramenta junto do prompt, e ele responde com base naquilo. Ninguém mais faz isso na escala dele.

Guiar passo a passo com paciência. Você diz "sou iniciante, me guie", e ele realmente segue. Para. Explica. Espera confirmação. As outras IAs tendem a "adiantar" e te dar 10 passos de uma vez. Gemini aceita ir devagar quando você pede.

Analisar planilhas e documentos longos. Jogou um CSV de 10.000 linhas? Ele analisa. Jogou um PDF de 200 páginas? Ele resume, extrai, responde perguntas. Pra isso, não tem igual.

Integração com Google (Drive, Docs, Sheets, Gmail). Se você vive dentro do ecossistema Google, Gemini é a escolha natural. Ele entende seus arquivos sem você precisar fazer upload manual toda vez.

O que Gemini NÃO faz tão bem

- **Escrita técnica de longa forma.** Ele escreve, mas o estilo não tem o mesmo peso editorial do Claude.
- **Código complexo multi-arquivo.** Ele tenta, mas se confunde mais que Claude em projetos grandes.
- **Criatividade livre.** Ele tende a ser mais "wikipedia" que "artista". Pra brainstorm criativo, use GPT.
- **Tarefas autônomas de ponta a ponta.** Ele responde, mas não "faz pra você". Pra isso você precisa de Manus.

A técnica "Me Guie Passo a Passo" — o pulo do gato #1

Aqui está a razão pela qual Gemini vai ser seu melhor amigo nos próximos 30 dias.

Existe um formato de prompt que transforma Gemini em tutor paciente, passo a passo. Vou te dar o template exato agora. Decore. Este prompt vai te salvar dezenas de vezes durante a trilha:

```
Sou iniciante absoluto. Nunca configurei [ASSUNTO] na vida.
```

```
Preciso: [OBJETIVO CONCRETO]
```

```
Stack que estou usando: [LISTA]
```

```
Me guie passo a passo. Regras:
```

- Pergunte antes de assumir qualquer coisa.
- Valide cada etapa comigo antes de passar pra próxima.
- Quando eu tiver dúvida, vou mandar print da tela.
- Se eu errar algo, me diga como desfazer.
- Evite jargão técnico. Se precisar usar, defina em 1 frase.

```
Vamos começar pela etapa 1.
```

Esse prompt ativa o modo tutor. Gemini para de despejar 10 passos e vai com você um a um.

Quando você travar em alguma etapa, **tire um print da tela inteira** (janela do navegador + mensagens de erro, se houver) e cole direto na conversa. Gemini olha a imagem, entende onde você está, e te diz exatamente o que fazer em seguida.

5 situações em que Gemini vai salvar sua pele

Durante a trilha de 30 dias, você vai precisar do "me guie passo a passo" do Gemini em pelo menos estas situações:

1. **Configurar Supabase do zero** — criar projeto, tabelas, RLS, auth.
2. **Apontar domínio e configurar DNS** — o campo mais traumático do iniciante.
3. **Ativar webhook do Stripe** — crítico, sem isso pagamento não processa.
4. **Configurar OAuth com Google** — labirinto no Google Cloud Console.
5. **Fazer deploy no Vercel e ler logs de erro** — interpretar build falho.

Cada uma dessas situações é um capítulo da Parte 4. Mas o prompt-chave é o mesmo. Varia só o objetivo.

Uma sacada extra: Gemini dentro do Antigravity

Se você usa Antigravity (ferramenta desktop do Google que integra Claude Code + Gemini num único lugar), você tem um bônus: o Gemini enxerga o ambiente do seu projeto. Ele pode ler seus arquivos locais, seus logs, seu terminal, e te guiar com o contexto inteiro do projeto à mão.

Não é pré-requisito. Mas se você gosta de ter tudo num lugar só, vale instalar.

Capítulo 5 — GPT, o Sparring Criativo

GPT (ChatGPT da OpenAI) é a ferramenta que você usa quando precisa **pensar rápido, criar em volume, e iterar sem peso**.

Se Claude é o engenheiro introvertido e Gemini é o professor paciente, GPT é aquele colega hiperativo que solta 40 ideias por minuto. Nem todas são boas. Mas é com ele que você destrava o bloqueio criativo.

O que GPT faz melhor que os outros

Brainstorm de volume. "Me dá 50 opções de nome pro meu produto." GPT entrega sem reclamar. Claude provavelmente entrega 10 com uma dissertação. Gemini entrega 30 genéricas. GPT consegue ser solto, ousado, variado.

Copy punchy. Pra headline de landing, legenda de ad, roteiro de reel, thread de X — GPT tem ritmo. Claude é mais "correto". GPT é mais "vende". Em marketing, vender ganha de correto.

Imagens com DALL-E. Essa é exclusividade. Nem Claude nem Gemini (pelo menos não da mesma forma integrada) geram imagens. GPT + DALL-E te entrega assets visuais direto no chat.

GPTs customizados. Você pode criar seu próprio GPT especializado (ex: "Copywriter da minha marca") e reusar sempre. Claude tem Projects pra coisa parecida, mas o sistema de GPTs customizados é mais maduro e tem marketplace próprio.

Ações rápidas. Você joga uma pergunta, ele responde em segundos. Pra ajustes rápidos de texto, renomeações, variações criativas — GPT é mais ágil.

O que GPT NÃO faz tão bem

- **Código em projeto longo.** Alucina mais que Claude quando o contexto cresce.
- **Análise técnica profunda.** Tende a "performar inteligência" sem de fato raciocinar até o fim.
- **Escrita editorial de peso.** O texto tem energia, mas falta densidade.
- **Tutoriais passo a passo pacientes.** Ele adianta. Pra iniciante, pode assustar.
- **Contexto ultra longo.** Se você joga 50 páginas de doc, ele começa a esquecer partes.

Quando chamar GPT no método

Durante a trilha, você vai abrir o GPT principalmente em três momentos:

1. **Fase 1 — Idear.** Brainstorm de nome, ângulos de posicionamento, variações criativas.
2. **Fase 3 — Construir.** Gerar imagens pra landing, ícones, assets visuais.
3. **Fase 7 — Vender.** Copy de landing, variações de anúncio, emails.

Nas outras fases, GPT pode ser útil também, mas o protagonismo fica com Claude (arquitetura) e Gemini (configuração).

Prompt-chave pra GPT em brainstorm

Gere 30 opções de [NOME / HEADLINE / ÂNGULO] pro seguinte produto.

PRODUTO: [1 frase]

PÚBLICO: [quem compra]

SENSAÇÃO DESEJADA: [premium / jovem / sóbrio / descontraído]

Divida em 5 categorias, 6 opções cada:

1. [categoria 1 - ex: nomes inventados tipo Uber, Kavak]
2. [categoria 2 - ex: combinações de palavras tipo DigitalOcean]
3. [categoria 3 - ex: metáforas poéticas tipo Arc, Linear]
4. [categoria 4 - ex: nomes em português que não soam brega]
5. [categoria 5 - ex: siglas ou abreviações curtas]

Note que o prompt impõe estrutura. Sem estrutura, GPT dispara em todas as direções. Com estrutura, ele dispara onde você quer.

Sobre os GPTs customizados

Se você usa ChatGPT Plus, considere criar um GPT customizado chamado "Copywriter [seu produto]" com instruções tipo:

Você é copywriter especializado em infoprodutos BR.
Tom: direto, brasileiro, anti-hype. Zero "transforme sua vida".
Sempre que eu pedir copy, entregue 3 variações diferentes em
ângulo (dor / desejo / prova). Max 4 linhas cada.

Aí toda vez que você precisa de copy, abre esse GPT. Consistência sem copiar e colar regras.

Capítulo 6 — Lovable, o Executor Visual

Lovable é a ferramenta que tira seu projeto do papel em minutos. Você escreve um prompt descrevendo o que quer, e ele te entrega um aplicativo React rodando, com interface, rotas, componentes — tudo funcional.

É o instrumento mais espetacular das cinco. E também o mais perigoso, se usado errado.

O que Lovable faz melhor que os outros

Scaffold visual imediato. Você descreve um app e em 2 minutos tem um app. Com telas, navegação, design decente. Nenhuma outra ferramenta te dá esse "pulo" inicial com tão pouco atrito.

Preview em tempo real. Toda vez que você muda algo, o preview atualiza. Você vê o que está construindo enquanto constrói. Isso elimina a "cegueira" de programar sem ver o resultado.

Deploy automático. Lovable hospeda seu app numa URL pública automaticamente. Você pode mandar o link pra alguém testar em 5 minutos de projeto.

Integração com Supabase nativa. Com 2-3 cliques, você conecta Lovable ao Supabase, e ele já gera código pra auth, queries, inserts, etc. Não é magia — é integração bem feita.

GitHub sync. Ele sincroniza automaticamente com um repositório GitHub. Isso abre o "pulo do gato" que você vai ver na Parte 4.

O que Lovable NÃO faz bem

Aqui está a parte que quase ninguém fala:

- **Edição iterativa é cara.** Cada prompt que você manda dentro do Lovable queima crédito. Projeto médio com muitas iterações pode queimar 50-100 dólares facilmente.
- **Refatorações complexas.** Quando o projeto cresce, pedir mudanças estruturais dentro do Lovable fica lento e caro.
- **Debugging profundo.** Se o código gerado tem bug, corrigir dentro do Lovable é exaustivo. Melhor tirar do Lovable e corrigir fora.
- **Customizações muito específicas.** Às vezes você quer mudar algo pequeno e Lovable insiste em mudar coisas adjacentes também.

A regra sagrada do Lovable

Esta é a frase mais importante deste capítulo:

Lovable é pra COMEÇAR, não pra continuar.

Use Lovable pra gerar o scaffold inicial (1 a 3 prompts). Depois disso, saia dele. Continue o desenvolvimento no Cursor ou Claude Code (Parte 4 explica). Use Lovable só como visualizador do progresso.

Essa regra sozinha pode te economizar 300 dólares nos próximos 30 dias. Não é exagero.

O prompt de scaffold que economiza 80% de crédito

A maioria das pessoas perde crédito no Lovable porque manda prompts curtos e vai refinando. Dez prompts pequenos queimam mais crédito que um prompt bem feito grande.

Template do prompt de scaffold perfeito:

```
Crie um aplicativo web com as seguintes especificações.

NOME: [nome]
DESCRIÇÃO: [1 frase do que faz]
STACK: React + TypeScript + Tailwind + shadcn/ui + Supabase

TELAS OBRIGATÓRIAS:
1. Landing pública (hero, features, pricing, footer)
2. Login / Signup via Supabase Auth (email + Google)
3. Dashboard protegido em /app
4. [tela principal 1]
5. [tela principal 2]

SCHEMA SUPABASE INICIAL:
[cole o schema que Claude te deu antes]

ESTÉTICA: [descreva em 3 palavras — ex: "dark, minimal, tipografia sóbria"]
INSPIRAÇÃO VISUAL: [cite 1-2 marcas]

Quero o scaffold completo funcional. Configure Supabase Auth,
proteja as rotas, integre o schema. DEPOIS disso vou mover o
projeto pro GitHub e editar fora do Lovable via Cursor/Claude Code.
```

Note o último parágrafo. Você está informando o Lovable que vai sair dele. Isso ajuda ele a não superproduzir.

Os 3 únicos momentos em que você volta pro Lovable

Depois do scaffold inicial, você volta ao Lovable só em três situações:

1. **Pra visualizar o preview.** Depois que você editou no Cursor/Claude Code e fez push, o Lovable atualiza o preview. Você vê o resultado no navegador dele.
2. **Mudanças triviais de UI puramente visuais.** Trocar a cor de um botão, ajustar padding — às vezes é mais rápido no Lovable do que abrir o editor.
3. **Compartilhar preview com cliente ou stakeholder.** A URL do Lovable é pública e fácil de compartilhar.

Tudo fora disso: fica no editor.

O que o Lovable te ensina (sem querer)

Mesmo que você não saiba programar, só de ler o código que o Lovable gera você começa a pegar padrões:

- Estrutura de componentes React.
- Como hooks funcionam na prática.
- Como queries do Supabase são escritas.
- Como routes são protegidas.

Você não precisa dominar isso. Mas depois de 30 dias usando o método, você vai **entender** o que está acontecendo. E isso é mais do que 90% das pessoas que "aprenderam React" na marra.

Capítulo 7 — Manus, o Operário Autônomo

Manus é a ferramenta mais diferente das cinco. Porque com ela você não **prompta** — você **delega**.

As outras quatro são assistentes: você pergunta, elas respondem. Manus é agente: você define uma tarefa, ele sai, executa, volta com o resultado pronto. A diferença operacional é gigante.

O que Manus faz melhor que os outros

Executar tarefas de longa duração autonomamente. Você diz "faça uma pesquisa de mercado profunda sobre [nicho] e me entregue um relatório executivo em PDF". Manus sai. Pesquisa. Cruza fontes. Gera gráficos. Formata. Entrega PDF pronto depois de 30-60 minutos. Você não supervisiona cada passo.

Produzir entregáveis finais, não rascunhos. Quando Manus termina, você tem um arquivo pronto. Um PDF, uma apresentação, uma planilha, um blog post completo. Não é "base pra você trabalhar em cima" — é entregável.

Tarefas multi-passo complexas. Operações que envolvem vários "subtarefas" encadeadas (pesquisar → analisar → comparar → compilar → formatar) são onde Manus brilha. Nas outras ferramentas, você teria que orquestrar cada passo manualmente.

Processar grandes volumes. Produzir 30 posts de blog em batch. Analisar uma planilha de 50.000 linhas. Fazer due diligence técnica profunda de uma ferramenta. Tudo isso Manus faz bem.

O que Manus NÃO faz bem

- **Decisões estratégicas finas.** Ele executa bem, mas decidir "fazer X ou Y" é melhor pro Claude.
- **Iteração criativa rápida.** Cada rodada do Manus é lenta. Pra brainstorm ágil, use GPT.
- **Trabalho que exige você no loop.** Se é coisa que muda a cada 5 minutos com o feedback, delegar pro Manus é frustrante.
- **Precisão matemática de cálculos complexos.** Ele tenta, mas pode errar em contas delicadas. Valide.

A mudança de mentalidade: delegar vs. promptar

Pra usar Manus bem, você precisa virar a chave no cérebro.

Promptar é: você está presente, conversando, iterando. "Escreva isso, agora aquilo, agora ajuste aquilo."

Delegar é: você descreve o resultado final esperado com o máximo de detalhe, manda, e volta depois. "Faça X, com essas características, no formato Y, entregue em Z."

A qualidade do que Manus entrega é **proporcional à qualidade do briefing**. Briefing ruim → entrega ruim. Briefing bem feito → entrega de 8 horas humanas em 40 minutos.

Quando chamar Manus no método

As tarefas que se beneficiam de Manus durante a trilha:

1. **Fase 1 — Idear.** Pesquisa profunda de mercado com relatório pronto. Análise de dados se você tem planilha.
2. **Fase 2 — Especificar.** Due diligence técnica da stack (esperado vs. realidade de cada ferramenta).
3. **Fase 7 — Vender.** Produção de conteúdo em batch (30 posts, 15 reels, sequência de email).

Fora disso, use as outras ferramentas.

O briefing perfeito pro Manus

Um briefing bem feito pro Manus tem 4 partes:

1. **Objetivo final.** Não "faça pesquisa" — "entregue relatório de 15-25 páginas sobre [X]".
2. **Contexto.** Quem é você, pra que vai usar isso, quem vai ler.

3. Formato do entregável. PDF executivo? Planilha? Markdown? Quantas páginas/linhas/posts?

4. Restrições e critérios. Todas as afirmações com fonte linkada. Fontes primárias preferidas. Tom executivo. Sem jargão. Etc.

Exemplo concreto de briefing bem feito:

```
Faça pesquisa de mercado completa. Entregue relatório em PDF.

NICH0: SaaS de gestão financeira pra micro empreendedores BR.
REGIÃO: Brasil.
AUDIÊNCIA DO RELATÓRIO: eu + 2 sócios técnicos.

FORMATO: PDF de 20 páginas, design executivo, tabelas e gráficos.

CONTEÚDO OBRIGATÓRIO:
1. Tamanho do mercado (dados com fonte)
2. 30 concorrentes com URL, preço, proposta de valor
3. Matriz SWOT dos top 5
4. 5 tendências dos últimos 12 meses com evidência
5. 3 lacunas não atendidas
6. 5 ângulos de posicionamento diferenciado
7. Apêndice com fontes linkadas

TOM: executivo, direto, sem floreio.
FONTES: priorizar dados primários (Statista, reports setoriais,
relatórios de empresas listadas). Nada de blog genérico.
```

Esse briefing produz um PDF que 3 consultorias cobrariam 15.000 reais pra entregar em 3 semanas. Manus faz em 40 minutos.

A validação obrigatória

Uma coisa crítica: **Manus alucina**. Às vezes inventa URLs que não existem, atribui quotes a pessoas que não disseram aquilo, cita dados com precisão falsa.

Regra: **antes de usar qualquer dado do relatório dele em algo público, valide 3-5 fontes críticas manualmente**. Abre os links. Confere os números.

Manus é um acelerador brutal, mas não é infalível. Seu papel como maestro continua: validar o que chega.

Síntese da Parte 2 — A partitura das cinco ferramentas

Se você memorizar uma única coisa da Parte 2, seja esta tabela:

Ferramenta	Papel	Use quando
Claude	Arquiteto	Planejar, especificar, auditar, decidir
Gemini	Tutor	Configurar coisas que você não sabe fazer
GPT	Sparring	Brainstorm rápido, copy, imagem
Lovable	Executor visual	Scaffold inicial e preview
Manus	Operário autônomo	Delegar tarefas longas com entregável pronto

Cole isso num papel. Deixa do lado do computador. Nos próximos 30 dias, toda vez que você for abrir uma aba, olhe a tabela primeiro.

Na Parte 3, vamos juntar tudo: o ciclo dos 5 passos que você vai rodar a cada fase da trilha.

PARTE 3 — O Método em Ação

Até aqui, você tem as ferramentas e o mindset. Agora vem o fluxo operacional. Esta parte é a receita.

Capítulo 8 — O Ciclo dos 5 Passos

Antes de entrar nos estudos de caso, precisamos apresentar o núcleo operacional do método: o **Ciclo dos 5 Passos**.

Este ciclo é a sequência que você vai rodar dentro de cada fase da trilha. Não é "a ordem das fases" (essa é a trilha, capítulo 15 em diante). É o padrão de pensamento dentro de qualquer tarefa não-trivial que você for fazer.

Depois que você internaliza o ciclo, ele vira automático. Toda tarefa nova, você pensa: "em qual passo estou agora?". Isso sozinho já resolve 80% do caos que a gente discutiu no Capítulo 1.

Os 5 Passos

- | | |
|--------------|-----------------------------|
| 1. IDEIA | → Claude |
| 2. PESQUISA | → Gemini ou Manus |
| 3. PLANO | → Claude |
| 4. EXECUÇÃO | → Lovable, Claude Code, GPT |
| 5. AUDITORIA | → Claude |

Cada passo tem uma ferramenta principal. Cada passo tem um entregável concreto. E cada passo só pode começar depois que o anterior está fechado.

Vou explicar um a um.

Passo 1 — IDEIA (Claude)

Objetivo: sair de "tenho uma ideia vaga" pra "tenho um problema definido, com persona e proposta de valor clara".

Ferramenta principal: Claude.

Por que Claude: porque ideia vaga precisa de raciocínio estruturado pra virar algo concreto. Claude pressiona a ideia, aponta falhas, sugere variações. Ele é o "investidor cético" que você precisa nessa hora.

Entregável: um parágrafo único, de no máximo 5 frases, que responde:

- O que é.
- Pra quem é.
- Que problema resolve.
- Qual é o diferencial.
- Como ganha dinheiro.

Se você não consegue condensar sua ideia em 5 frases, você não tem ideia ainda — tem desejo. Volta pro Claude e pressiona mais.

Passo 2 — PESQUISA (Gemini ou Manus)

Objetivo: descobrir o que já existe, o que o mercado precisa e o que falta.

Ferramenta principal: Gemini pra pesquisa rápida + leitura de dados. Manus pra pesquisa profunda com relatório formatado.

Por que dividir: Gemini é ótimo pra "vasculhada rápida" em 15-30 minutos. Manus é pra "mergulho profundo de 40-60 minutos com PDF pronto no final". Você escolhe a profundidade conforme o momento.

Entregável: uma tabela de concorrentes (mínimo 10), um parágrafo sobre o tamanho de mercado, e 3 ângulos de diferenciação que você pode explorar.

Regra de ouro: não pule este passo achando que "já conhece o mercado". Você sempre acha que conhece. Sempre tem algo que escapou. Em 2 horas de pesquisa bem feita, você descobre que uma ferramenta que já resolve seu problema existe há 4 anos e fatura 2 milhões. Melhor descobrir agora do que depois de 30 dias construindo.

Passo 3 — PLANO (Claude)

Objetivo: transformar a ideia + pesquisa em um plano de execução detalhado. Isso é o PRD + o schema + a lista de tarefas.

Ferramenta principal: Claude (em um Project, pra manter contexto).

Por que Claude: porque plano bom precisa de coerência interna e raciocínio técnico. É onde Claude mais brilha.

Entregável: três documentos — PRD preenchido, schema Supabase em SQL, e lista das 10-15 tarefas principais pra construir o MVP em ordem de execução.

Regra: você não passa pro passo 4 sem os três documentos prontos. Construir sem plano é construir em círculos. Vai gastar dinheiro, queimar crédito e não chegar em lugar nenhum.

Passo 4 — EXECUÇÃO (Lovable + Claude Code + GPT)

Objetivo: transformar o plano em código rodando.

Ferramentas principais:

- **Lovable** pro scaffold inicial (1 a 3 prompts).
- **Claude Code** (via terminal, no seu computador) pra todas as edições depois do scaffold.
- **GPT** pontualmente pra gerar imagens e copy.

Por que essa divisão: é o "pulo do gato" completo — você aproveita o melhor do Lovable (scaffold rápido) sem cair na armadilha do Lovable (edição cara). Claude Code faz o que Lovable faria, mas com uma fração do custo. GPT entra só pra tarefas criativas rápidas.

Entregável: MVP funcional, no preview do Lovable, conectado ao GitHub, com o código sendo editado fora do Lovable. Auth funcionando. Telas principais navegáveis.

Passo 5 — AUDITORIA (Claude)

Objetivo: revisar o que foi construído antes de subir pra produção.

Ferramenta principal: Claude, em auditoria estruturada.

Por que Claude: porque auditoria exige olhar crítico, identificar bug sutil, encontrar brecha de segurança, apontar código duplicado. Claude enxerga esses problemas que o iniciante não vê.

Entregável: lista de achados em 4 categorias (bugs / segurança / performance / manutenibilidade), com correções sugeridas. Depois: versão corrigida.

Regra: nenhum código vai pra produção sem passar por este passo. É inegociável. Se você pular, vai ter brecha de RLS, senha exposta, bug no fluxo de pagamento. E aí você processa reembolso.

O ciclo dentro da trilha

Este ciclo de 5 passos se repete em cada fase da trilha. Nem sempre você passa pelos 5 — às vezes a fase só precisa de Execução + Auditoria (ex: Fase 6 Publicar). Mas o padrão está sempre ali.

Pense assim:

- **Fase 1 (Idear)** = rodar os passos 1 e 2 intensamente.
- **Fase 2 (Especificar)** = rodar o passo 3.
- **Fase 3 (Construir)** = primeira rodada dos passos 4 e 5.
- **Fase 4 (Integrar)** = segunda rodada dos passos 4 e 5, com foco em auth e pagamento.
- **Fase 5 (Polir)** = auditoria pesada (passo 5) + correções (passo 4).
- **Fase 6 (Publicar)** = execução final (passo 4).
- **Fase 7 (Vender)** = novo ciclo começando — mas agora pra marketing, não produto.

O ciclo é uma lente. A trilha é o caminho. Você usa a lente pra enxergar cada pedaço do caminho.

O erro mais comum: pular o passo 3

Iniciantes fazem isso o tempo todo. Tem uma ideia, dá uma pesquisada rápida, e já abre o Lovable. Pula o PRD, pula o schema, pula o plano.

O que acontece: você prompta no Lovable sem saber exatamente o que quer. Lovable entrega algo. Você vê e percebe que não era bem aquilo. Prompta de novo. Queima crédito. Itera. Itera. 30 dólares depois, o que você tem é uma tela sem rumo e um saldo no cartão.

Com o plano, você entra no Lovable sabendo exatamente: "quero 5 telas específicas, com esses componentes, conectadas a esse schema, com essa estética". Lovable entrega certo. 1-2 prompts e o scaffold está pronto.

A diferença entre "gastei 3 dólares e tenho o MVP" e "gastei 60 dólares e não tenho nada" é **uma hora de plano com Claude**.

Dedique essa hora. Sempre.

Capítulo 9 — Estudo de Caso 1: Do zero ao PRD

Vamos ver o ciclo funcionando na prática. Pra este estudo de caso, vou usar um exemplo concreto — um produto hipotético chamado "Foconote". Observe o padrão mais do que os detalhes do produto em si.

Contexto

Ideia inicial (vaga): "Acho que existe espaço pra um app de notas que ajuda pessoas com TDAH a focar."

Note que essa ideia tem tudo de errado: é vaga, assume mercado sem verificar, não tem diferencial, não tem modelo de receita. É o ponto de partida realista de quase todo empreendedor.

Passo 1 — Ideia (Claude)

Prompt enviado ao Claude:

Aja como investidor cético. Vou descrever uma ideia de SaaS. Sua tarefa: listar os 5 maiores riscos em ordem de gravidade, identificar 3 suposições que estou assumindo como verdade, e sugerir o menor experimento possível para validar a suposição mais crítica. Se a ideia for fraca, fale sem rodeios.

IDEIA: Um app de notas feito pra pessoas com TDAH, com features específicas pra ajudar a focar. PÚBLICO: Pessoas com TDAH diagnosticado ou suspeita, 22-45 anos, que trabalham com atividade intelectual e têm dificuldade de produtividade. MODELO DE RECEITA: Assinatura mensal R\$ 19,90.

Claude respondeu com os 5 riscos, as suposições e um experimento. Resumidamente:

Riscos apontados:

1. Notion e Obsidian já dominam — diferenciação insuficiente.
2. Suposição de que TDAH precisa de "features específicas" pode ser ilusão.
3. Ticket baixo (R\$ 19,90) exige escala pra ser viável — CAC vai comer margem.
4. Público fragmentado: "TDAH" é diagnóstico, não segmento comercial.
5. Barreira técnica baixa — qualquer concorrente replica em 3 meses.

Suposições não validadas:

1. Pessoas com TDAH abandonam ferramentas genéricas *por falta de features específicas* (pode ser por outro motivo).
2. Existe disposição a pagar por feature vertical quando Notion é grátis.
3. O volume de pessoas com TDAH buscando solução é grande o suficiente.

Experimento sugerido: Landing simples explicando o produto imaginado. Anúncio no Instagram/TikTok segmentando palavras-chave de TDAH. Observar CTR e cadastros em waitlist. Meta: 3% de CTR e 100 cadastros em 7 dias com R\$ 200 de budget. Se bater, valida desejo. Se não, a ideia precisa ser repensada.

O que fizemos com essa resposta: Não seguimos cegamente. Fizemos o experimento. Bateu 1.2% de CTR (metade do esperado) e 30 cadastros (um terço). Resultado: reformulação.

Nova ideia refinada:

Ferramenta de captura rápida de ideias fragmentadas pra profissionais criativos (designers, escritores, devs) que trabalham em sprints curtos de foco e precisam despejar pensamentos sem organizar. Diferente de Notion porque não tem estrutura — é como um "caderninho digital sempre aberto".

Note: a persona mudou. O problema mudou. O ângulo mudou. O formato do produto ficou mais específico. Isso é validação funcionando.

Passo 2 — Pesquisa (Gemini + Manus)

Pesquisa rápida com Gemini:

Faça uma varredura rápida dos concorrentes de capturadores rápidos de ideias pra pessoas criativas. Foco em:

- Drafts (iOS)
- Bear
- Apple Notes
- Google Keep
- Apps de "second brain" tipo Capacities, Tana. Entregue: tabela com proposta de valor, público, preço, ponto fraco. Só entregue dados que tiverem fonte verificável.

Gemini retornou tabela com 8 concorrentes. Principais achados:

- Drafts é o mais similar — mas é só iOS.
- Bear tem foco em escrita, não captura rápida.
- Google Keep é grátis — barreira de preço enorme.
- Nenhum deles tem integração com IA pra "destilação" da ideia depois.

Insight: o ângulo "captura rápida + IA organiza depois" era lacuna real.

Pesquisa profunda com Manus (delegada, voltou 45 min depois com PDF):

Faça pesquisa profunda do mercado brasileiro de ferramentas de produtividade pessoal. Entregue: relatório de 20 páginas cobrindo tamanho, concorrentes (BR e global), tendências 2024-2026, ticket médio disposto, canais de aquisição dominantes neste nicho. Todas as afirmações com fonte linkada.

Manus entregou PDF com:

- Mercado BR de produtividade pessoal: USD 340M, crescendo 18% aa.
- 15 concorrentes BR (vários desconhecidos até então).
- Canal dominante de aquisição: YouTube (via influenciadores de produtividade).
- Ticket médio do nicho: R\$ 29/mês. R\$ 19,90 estava subprecificado.
- Tendência forte: "AI-powered" é diferencial ativo em 2026, mas em 12 meses vira commodity.

Entregável do passo 2: arquivo `pesquisa-mercado.md` com as duas análises consolidadas.

Passo 3 — Plano (Claude em Project)

Criado Project no Claude chamado "Foconote". Upload do PRD template e da pesquisa de mercado no contexto do projeto.

Prompt:

Com base no template PRD e na pesquisa anexada, escreva o PRD completo pra versão 1.0 do Foconote. Stack: Lovable + Supabase

- Stripe + Resend. Preço: R\$ 29/mês. Público: profissionais criativos. Foco do MVP em captura rápida + destilação via IA. Seja específico, use o template como guia, mas corte features que não são essenciais pro MVP.

Claude entregou PRD de 14 páginas. Principais decisões de escopo:

ENTRA no MVP:

- Captura rápida via atalho de teclado (universal no app).
- 3 tipos de "destilação" via IA: resumo, lista de ações, expansão.
- Tag automática (IA sugere).
- Busca full-text.
- Conta e assinatura Stripe.

FICA DE FORA:

- Colaboração em equipe.
- App mobile nativo (só web responsivo na v1).
- Integrações com outros apps (só export em markdown).
- Categorias manuais (só tags automáticas).

Schema Supabase gerado: Tabelas `profiles`, `subscriptions`, `notes`, `distillations`, `tags`. Total 5 tabelas. RLS configurado em todas.

Entregáveis finais do passo 3:

- `prd-foconote-v1.md` (14 pág)
- `schema-foconote.sql` (pronto pra colar no Supabase)
- `tarefas-construcao.md` (12 tarefas em ordem)

Por que parar por aqui neste capítulo

Este estudo de caso foca nos passos 1 a 3 — da ideia ao PRD. É a parte **estratégica**. O próximo capítulo mostra os passos 4 e 5 do mesmo exemplo: construir o MVP e auditar antes de subir.

A lição central deste capítulo: **a primeira ideia quase nunca é a ideia que vai pra produção**. O ciclo de 5 passos força você a pressionar a ideia, pesquisar o mercado, e replanejar antes de escrever qualquer linha de código.

Isso é o oposto do que o iniciante típico faz (ideia → código). E é por isso que o iniciante típico não lança.

Capítulo 10 — Estudo de Caso 2: Do PRD ao MVP no ar

Continuamos com o Foconote. PRD pronto, schema pronto, tarefas listadas. Vamos construir.

Passo 4 — Execução (parte 1: scaffold no Lovable)

Com o PRD em mãos, o prompt enviado ao Lovable foi UM SÓ, bem calibrado:

Crie um aplicativo web com as seguintes especificações.

NOME: Foconote DESCRIÇÃO: Ferramenta de captura rápida de ideias com destilação via IA. STACK: React + TypeScript + Tailwind + shadcn/ui + Supabase.

TELAS OBRIGATÓRIAS:

1. Landing pública (hero, features, pricing, CTA)
2. Login / Signup via Supabase Auth (email + Google)
3. /app — editor principal com sidebar de notas + área de texto
4. /app/settings — perfil e assinatura

SCHEMA SUPABASE: [colado aqui o SQL completo do passo 3]

ESTÉTICA: dark minimal, tipografia sóbria, ênfase em tipografia (Fraunces + Inter), paleta bege/grafite. Inspiração: Linear, Arc.

Configure Supabase Auth, proteja /app/ e /app/settings/*, integre o schema. DEPOIS disso vou mover o projeto pro GitHub e editar fora do Lovable via Claude Code.*

Resultado: scaffold em 3 minutos. 4 telas navegáveis. Auth funcionando com placeholder. Schema conectado. Custo: 1 prompt, cerca de 25 centavos de crédito.

Um segundo prompt pra refinar:

Ajuste o editor principal: remova a sidebar de categorias, adiciona atalho global Cmd+N pra nova nota, e adiciona dropdown "destilar" com 3 opções (resumir, extrair ações, expandir).

Pronto. Segundo e último prompt no Lovable. Daqui pra frente, tudo foi no Claude Code.

Passo imediato depois:

1. Conectar Lovable ao GitHub (botão no próprio Lovable).
2. Clonar repo localmente.
3. Abrir terminal na pasta, rodar `claude` pra abrir Claude Code.

Status: projeto rodando, scaffold completo, saindo do Lovable.

Passo 4 — Execução (parte 2: edições no Claude Code)

Aqui começa a economia real. Todas as edições a partir daqui acontecem no Claude Code, não no Lovable.

Exemplo de prompt dentro do Claude Code:

Este projeto é um capturador de notas. Quero implementar a função "destilar". O fluxo é:

1. Usuário seleciona uma nota.
2. Clica em "destilar" e escolhe o tipo (resumir, ações, expandir).
3. Frontend manda POST pra Edge Function do Supabase.
4. Edge Function chama API da Anthropic (Claude Sonnet) com o prompt específico do tipo.
5. Resposta volta e é salva na tabela `distillations`.
6. Frontend exibe a destilação ao lado da nota original.

Preciso da Edge Function completa, o código do frontend pra chamar, e as tipagens TypeScript. Antes de mexer nos arquivos, me liste o que vai alterar e peça aprovação.

Claude Code listou 4 arquivos a editar, pediu aprovação, editou, commitou. Preview no Lovable atualizou em 30 segundos.

Custo estimado dessa tarefa: 1-2 centavos de uso de API da Anthropic. A mesma tarefa no Lovable teria custado 8-15 prompts (pedir, ajustar, testar, corrigir), ou seja, cerca de 2-4 dólares.

Esse é o tamanho da economia. Por tarefa. Multiplicado por 40-60 tarefas ao longo de 30 dias.

Passo 4 — Execução (parte 3: assets visuais com GPT)

Pra landing, precisávamos de 3 elementos visuais: hero illustration, ícones pras 3 features principais, e uma imagem OG pra compartilhamento em redes sociais.

Prompt pro GPT + DALL-E:

Gere uma ilustração hero pra landing de um app chamado Foconote. Estilo: isométrico minimalista, paleta bege e grafite, foco numa folha de caderno flutuante com pontos de luz sugerindo ideias sendo capturadas. Dimensão 16:9. Sem texto. Estética: Linear, Notion.

3 iterações. Imagem aprovada. Otimizada no TinyPNG. Subida pro projeto.

Total de tempo no GPT: 15 minutos. 3 assets prontos.

Passo 5 — Auditoria (Claude)

Antes de publicar em produção, rodamos o passo 5: auditoria profunda do código gerado.

Prompt no Claude (com arquivos anexados):

Audite estes 4 arquivos do projeto Foconote. Use estrutura:

1. BUGS que vão quebrar
2. SEGURANÇA (RLS, inputs, chaves expostas)
3. PERFORMANCE (queries N+1, re-renders, missing indexes)
4. MANUTENIBILIDADE

Ordene do mais crítico ao menor. Para cada achado: gravidade, linha, correção em código.

Claude encontrou:

Críticos (3):

- Edge Function não valida se user está autenticado antes de chamar API da Anthropic. Qualquer requisição HTTP externa poderia consumir crédito.
- `stripe-signature` não está sendo validado no webhook — brecha grave.
- Campo `email` do profile não tem unique constraint — possível duplicata.

Médios (5):

- Queries na tela principal fazem N+1 (5 requests separados pra pegar tags).
- Re-render em cascata no editor principal ao digitar.
- Sem tratamento de erro na chamada da destilação — trava a UI.
- Chave pública do Supabase exposta no bundle (OK, é pública, mas não documentado).
- Tempo médio de carregamento: 3.1s. Meta 2.0s.

Baixos (4): detalhes de código.

Com essa lista, voltamos pro Claude Code e corrigimos um por um. Cada correção era um prompt específico ("aplicar correção 1", "aplicar correção 2", etc). Commits separados pra cada correção, pra ser fácil reverter se algo quebrar.

Depois das correções: segunda auditoria. 0 críticos. 1 médio (performance residual). Aceito.

Resultado final

- MVP funcionando em URL pública do Lovable.
- Auth funcionando (email + Google).

- Schema no Supabase com RLS validado.
- Edge Function de destilação funcionando com Claude Sonnet.
- Stripe integrado em test mode (ainda não live).
- Auditoria aprovada.

Custo total do Lovable: cerca de 40 centavos (2 prompts de scaffold + uns 3 ajustes triviais depois). **Custo total do Claude Code (API Anthropic):** cerca de 12 dólares em 3 semanas de edição. **Custo comparativo se tivesse feito tudo no Lovable:** estimados 150-200 dólares em créditos.

Economia real do método: ~90%.

Lição do estudo de caso

O que separa quem publica de quem não publica não é habilidade técnica. É seguir o ciclo. O Focote hipotético fez:

1. Ideia → pressionada com Claude → reformulada.
2. Pesquisa → concorrentes mapeados, oportunidade validada.
3. Plano → PRD, schema, tarefas — tudo antes de código.
4. Execução → scaffold rápido no Lovable, edição barata no Claude Code, assets no GPT.
5. Auditoria → erros críticos pegos antes de produção.

Tudo isso em 3 semanas de trabalho não-integral, por uma única pessoa, sem saber programar no sentido tradicional.

A Parte 4 agora vai aprofundar os **dois pulos do gato** que tornam esse fluxo possível — o Gemini como tutor e a arquitetura Lovable + GitHub + Claude Code.

PARTE 4 — Os Dois Pulos do Gato

Este é o centro gravitacional do método. Se você lê só uma parte deste ebook, que seja esta. Os dois pulos do gato pagam o livro cem vezes.

Capítulo 11 — Pulo do Gato #1: Gemini como Tutor

Todo iniciante trava em algum momento. Sempre. O momento é sempre o mesmo: quando precisa configurar algo que nunca configurou.

Configurar banco de dados. Apontar DNS. Ativar webhook. Configurar OAuth. Deploy em Vercel. Essas são as cinco paredes em que 90% dos iniciantes batem e desistem.

Este capítulo é sobre derrubar essas cinco paredes com uma única técnica.

O problema é psicológico antes de técnico

Quando você abre o painel do Supabase pela primeira vez, você vê 18 abas laterais, 40 botões, jargão por todo lado. Sua reação natural é travar. Porque o painel assume que você sabe o que é "RLS", "PostgREST", "Realtime", "Storage". Você não sabe. Nada ali faz sentido.

A solução popular é péssima: ler documentação. A doc do Supabase assume o mesmo nível de conhecimento que o painel. Você ia abrir, ler 10 páginas, não entender nada, fechar.

Outra solução popular também é ruim: vídeo no YouTube. Vídeo de 2023 mostra um painel que mudou em 2026. Você segue os passos, botão está em outro lugar, você trava, fecha.

A solução que funciona: **conversar com uma IA que olha o painel junto com você.**

É isso que o Gemini faz, e faz melhor que qualquer outra ferramenta. Você manda print do painel, diz "estou aqui, meu objetivo é esse, o que clico agora?". Ele responde explicando o que aparece na tela e qual botão apertar.

É como ter um professor particular de tecnologia, disponível 24h, que nunca se irrita, nunca julga sua pergunta e nunca te faz sentir burro.

A técnica "Me Guie Passo a Passo"

O prompt matriz — este é literalmente o único prompt que você precisa memorizar pra destravar qualquer configuração técnica:

```
Sou iniciante absoluto. Nunca configurei [ASSUNTO] na vida.

Preciso: [OBJETIVO CONCRETO]

Stack que estou usando: [LISTA]

Me guie passo a passo. Regras:
- Pergunte antes de assumir qualquer coisa.
- Valide cada etapa comigo antes de passar pra próxima.
- Quando eu tiver dúvida, vou mandar print da tela.
- Se eu errar algo, me diga como desfazer.
- Evite jargão técnico. Se precisar usar, defina em 1 frase.
- Ao final de cada etapa, me diga "confirmado?" antes de seguir.

Vamos começar pela etapa 1.
```

Decore isso. Cola num arquivo. Use sempre que travar.

Esse prompt faz 5 coisas importantes:

1. Desliga o "modo intelectual" do Gemini e liga o "modo tutor".
2. Força ele a ir devagar em vez de despejar 10 passos.
3. Obriga ele a validar entendimento antes de avançar.
4. Abre canal pra você mandar print da tela.
5. Define a responsabilidade: ele explica, você executa, ele valida.

Tutorial 1 — Configurar Supabase do zero

Vamos passar em detalhe pela primeira aplicação prática. Este é o cenário: você acabou de criar conta no Supabase, abriu o painel, e está paralisado.

Seu prompt inicial:

Sou iniciante absoluto. Nunca configurei banco de dados na vida.

Preciso: criar projeto no Supabase com as tabelas profiles, subscriptions, items. Preciso também configurar Row Level Security (RLS) e integrar Google OAuth.

Stack: React + TypeScript + Supabase.

Me guie passo a passo. [resto do prompt matriz]

Vamos começar pela etapa 1.

Gemini responde:

Beleza. Etapa 1: criar o projeto.

1. Acesse <https://supabase.com/dashboard>
2. Faça login (use GitHub se já tem conta — é mais rápido).
3. Clique em "New project" no canto superior direito.
4. Preencha:
 - Organization: deixe o default ou crie uma nova.
 - Name: dê um nome (ex: "meu-saas-v1").
 - Database password: gere uma senha forte e SALVE num gerenciador de senhas (você vai precisar depois).
 - Region: escolha a mais próxima de você. Pra Brasil, "South America (São Paulo)".
 - Plano: deixe Free (você pode mudar depois).

Clique em "Create new project" e espere 2 minutos enquanto o projeto é provisionado.

Confirmado? Me avisa quando estiver pronto e vamos pra etapa 2.

Você faz exatamente isso. Se trava em algo, tira print e cola. Gemini explica. Você destrava.

Etapa 2 (depois da confirmação):

Agora vamos criar as tabelas. Abre o painel do seu projeto e clica em "SQL Editor" no menu lateral.

Vou te passar o SQL completo, comentado, pra você colar e rodar. Copia o bloco abaixo:

[SQL com comentários explicativos]

Cola no editor, clica em "Run" no canto inferior direito. Se aparecer "Success. No rows returned", funcionou. Se aparecer erro, manda print.

Confirmado?

E assim por diante. Etapa a etapa. Quando precisar fazer algo no painel (não no SQL), Gemini explica qual botão, em qual menu, qual campo preencher.

Tempo total pra configurar Supabase completo com Gemini: 45 minutos a 1 hora, na primeira vez. Depois que você fez uma vez, as próximas levam 15 minutos.

Tempo pra configurar lendo doc sozinho: desiste no minuto 30 e vira "vou aprender programação primeiro".

Tutorial 2 — Apontar domínio e configurar DNS

Segunda parede clássica. Você comprou `meuproduto.com.br` no Registro.br, hospedou o site no Vercel, e agora precisa "apontar" o domínio. Não faz ideia do que isso significa.

Prompt:

```
Sou iniciante absoluto. Nunca configurei DNS na vida.
```

```
Preciso: apontar meu domínio meuproduto.com.br, comprado no  
Registro.br, pro meu site hospedado no Vercel.
```

```
[resto do prompt matriz]
```

```
Explique também o que cada tipo de registro DNS (A, CNAME, MX, TXT)  
significa antes de usar. Vamos começar pela etapa 1.
```

Gemini vai começar explicando:

- O que é DNS (em 2 frases).
- Por que existem vários tipos de registro.
- Qual você vai usar (normalmente A ou CNAME pra Vercel).

- Quanto tempo demora pra propagar.

Depois te guia:

1. No painel do Vercel: adiciona o domínio, copia os valores DNS que o Vercel mostra.
2. No painel do Registro.br: entra em "Configurações DNS" → "Editar zona".
3. Adiciona os registros exatamente como o Vercel pediu.
4. Salva.
5. Testa propagação em <https://dnschecker.org>.

Quando você trava em algum campo ("o que coloco no campo 'TTL?'"), tira print, manda, ele explica.

Tutorial 3 — Configurar webhook do Stripe

Terceira parede. Você integrou Stripe Checkout, o pagamento processa, mas... nada acontece depois. A assinatura não aparece no Supabase. O usuário paga e continua sem acesso premium.

O problema: falta configurar o webhook que avisa seu app que o pagamento foi feito.

Prompt:

```
Sou iniciante absoluto. Nunca configurei webhook de pagamento.
```

```
Preciso: configurar webhook do Stripe pra que, quando alguém  
assinar, meu app saiba e atualize a tabela `subscriptions` no  
Supabase.
```

```
Stack: Supabase Edge Functions, React, Stripe.
```

```
[resto do prompt matriz]
```

```
Explique também o que é webhook e por que eu preciso validar a  
assinatura cripto (stripe-signature) antes de confiar no payload.
```

Gemini vai:

1. Explicar o conceito de webhook em 2 frases.
2. Te guiar a criar uma Edge Function no Supabase.
3. Te dar o código completo da Edge Function (já com validação de assinatura).
4. Te guiar a criar o endpoint webhook no painel do Stripe.
5. Te ensinar a testar em sandbox com cartão de teste.

6. Te mostrar como checar logs de evento no painel do Stripe.

Esse é talvez o tutorial mais valioso dos cinco. Webhook mal configurado é onde 80% dos SaaS novos perdem dinheiro silenciosamente.

Tutorial 4 — Configurar OAuth com Google

Quarta parede. Você quer login com Google no seu app. Abre o Google Cloud Console pela primeira vez. Trava.

Prompt:

```
Sou iniciante absoluto. Nunca mexi no Google Cloud Console.  
  
Preciso: configurar "Sign in with Google" no meu app, integrado  
com Supabase Auth.  
  
Domínio: meuproduto.com.br  
Ambiente dev: localhost:3000  
  
[resto do prompt matriz]  
  
Aviso especial sobre redirect URIs — explique em detalhe porque é  
onde quase todo mundo erra.
```

Gemini vai:

1. Explicar a diferença entre Consent Screen e OAuth Credentials.
2. Te guiar a criar projeto no Google Cloud.
3. Te guiar a habilitar as APIs certas.
4. Criar credenciais OAuth Web Application.
5. Configurar redirect URIs (o ponto traiçoeiro — você precisa dos URIs de dev E produção).
6. Pegar client_id e client_secret.
7. Colar essas credenciais no painel do Supabase Auth.
8. Testar fluxo de login em dev.
9. Publicar consent screen pra usuários externos (sem isso, só suas contas internas conseguem logar).

Tutorial 5 — Deploy no Vercel e interpretação de logs

Quinta parede. Você tentou fazer deploy no Vercel, o build falhou, você olhou 400 linhas de log vermelho e fechou a aba.

Prompt:

```
Sou iniciante absoluto. Primeiro deploy no Vercel.  
  
Preciso: fazer deploy de produção do meu app React/TS hospedado  
no GitHub. Também preciso configurar variáveis de ambiente  
(VITE_SUPABASE_URL, VITE_SUPABASE_ANON_KEY, STRIPE_SECRET_KEY).  
  
[resto do prompt matriz]  
  
Se o build falhar, vou colar o log inteiro e você me explica  
o que significa e como corrigir.
```

Gemini vai:

1. Explicar o que é deploy e o que acontece nos bastidores.
2. Te guiar a conectar repositório GitHub ao Vercel.
3. Explicar cada campo na tela de configuração inicial.
4. Ensinar a diferença entre Preview e Production.
5. Cadastrar as variáveis de ambiente com o contexto certo (dev, preview, production).
6. Rodar primeiro deploy.

Quando o build falhar (e vai falhar na primeira vez, quase sempre), você cola o log inteiro no chat. Gemini lê, explica o que quebrou, te diz o que ajustar. Geralmente 2-3 iterações até o build passar.

Uma observação crítica sobre o Gemini

Em todos esses tutoriais, você está tratando o Gemini como tutor — não como solucionador automático. A diferença é sutil mas importante.

Tutor: explica, valida, ensina enquanto executa. Você entende o que está fazendo. **Solucionador automático:** faz pra você, você não aprende nada.

Você quer o primeiro. O Gemini faz bem o primeiro porque o prompt matriz força ele a esse comportamento. Sem o prompt matriz, ele cai no segundo modo, e aí você termina o processo com o DNS apontado mas sem ter entendido por quê.

Entender é valioso. Da próxima vez que precisar fazer algo parecido, você faz sozinho em 10 minutos.

Capítulo 12 — Pulo do Gato #2: Parte 1 — O Fluxo Econômico

Hora de falar sobre dinheiro.

Lovable é uma ferramenta maravilhosa. É também uma ferramenta cara. Não pelo preço de assinatura — mas pelo custo por edição.

Todo prompt que você manda dentro do Lovable queima créditos. Esses créditos são reais e acabam. Se você usa o Lovable da forma que parece natural (pedir, refinar, ajustar, iterar), você vai queimar dezenas de dólares numa semana, e possivelmente não vai ter produto publicado.

Este capítulo é sobre entender exatamente o tamanho do problema.

O que significa "edição iterativa"

Quando você constrói algo no Lovable, o fluxo natural é este:

1. "Crie um app de notas com login."
2. "Deixa o header mais escuro."
3. "Adiciona um botão no canto."
4. "Muda a fonte pra Inter."
5. "Move esse botão mais pra direita."
6. "Ah, e também quero que salve as notas no Supabase."
7. "Esqueci, adiciona autenticação com Google."
8. ...

Cada um desses 8 prompts queima crédito. Projeto real médio passa dos 60-80 prompts facilmente.

Cada prompt, em cima disso, regenera grandes porções do código. Lovable não é um editor de código — é um gerador de código. Cada prompt é uma nova geração quase completa.

A matemática do desastre

Vamos fazer uma simulação realista.

Cenário A — Iniciante sem método (100% Lovable):

- Scaffold inicial: 1 prompt.
- Ajustes visuais: 15 prompts.
- Adição de features: 25 prompts.
- Correções de bugs: 20 prompts.
- Integrações (Supabase, Stripe, Resend): 15 prompts.
- Ajustes finais pré-deploy: 10 prompts.

Total: cerca de 86 prompts no Lovable.

Pelo pricing do Lovable (que varia de plano, mas estima-se em USD 0.20 a USD 0.35 por prompt em projetos médios), isso gasta entre **USD 17 e USD 30** por semana de trabalho. Se o projeto demora 3 semanas, está falando em **USD 50 a USD 90** só em crédito Lovable.

Adiciona a assinatura mensal (USD 20-30) e você passou de USD 100 no projeto.

Cenário B — Iniciante com método (Lovable + GitHub + Claude Code):

- Scaffold inicial: 1 prompt no Lovable.
- Ajustes triviais visuais: 2 prompts no Lovable.
- **Tudo mais:** Claude Code.

Total no Lovable: 3 prompts. USD 1 a USD 1,50. **Total no Claude Code:** USD 20 por mês de assinatura (cobertura ampla).

Custo total do projeto no Cenário B: **USD 22**.

Diferença: **USD 80 economizados**. Isso é 400% mais barato.

Multiplique por vários projetos por ano. É o valor de um computador novo, ou uma viagem, ou o orçamento de ads pra lançar o produto.

Por que iterar no Lovable custa caro mesmo sendo "só um prompt"

Três razões técnicas:

1. Lovable regenera muito código a cada prompt. Ao pedir pra mudar a cor de um botão, Lovable pode regenerar o componente inteiro, o arquivo inteiro, ou até arquivos relacionados. Isso consome muito "token" por baixo — e custo sobe.

2. Lovable incorpora contexto longo. A cada prompt, Lovable lê o projeto inteiro (ou parte grande dele) pra entender o contexto. Projeto cresce, custo cresce.

3. Lovable é otimizado pra velocidade de criação, não eficiência de iteração. É um gerador de UI que virou editor de projeto. A arquitetura é boa pra gerar do zero, menos boa pra iterar.

Por que iterar no Claude Code custa muito menos

Claude Code é um CLI conversacional. Você roda no terminal, e ele edita arquivos do seu projeto local com base em prompts. Três vantagens:

1. Contexto controlado. Você (ou a IA) escolhe quais arquivos entram no contexto. Não carrega o projeto inteiro a cada prompt. Custa menos token por chamada.

2. Tarefas cirúrgicas. Você pode dizer "ajuste só a cor desse botão no arquivo X". Claude Code edita 1 linha. Custo: centavos.

3. Assinatura fixa vs. pay-per-prompt. O Claude Code Plan da Anthropic é uma assinatura mensal (a partir de USD 20/mês). Você usa bastante sem surpresa no cartão.

A única dúvida legítima: "mas Claude Code não é programar?"

Não. Ou melhor: é — mas no sentido "conversar com o código", não "digitar código".

No Claude Code você:

- Abre o terminal na pasta do projeto.
- Digita `claude` pra entrar no chat.
- Escreve em português: "ajuste a cor do botão principal pra bege" ou "adicione uma página de perfil com edição de dados".
- Claude Code edita os arquivos diretos.
- Você vê o diff (mudança), aprova ou pede ajuste.
- Commit + push, Lovable sincroniza o preview.

Você nunca escreveu uma linha de código "no dedo". Você só conversou.

É tão acessível quanto o Lovable. E 30x mais barato.

Matriz de decisão — quando usar o quê

Tarefa	Lovable	Claude Code
Scaffold inicial do projeto	✓	✗
Preview visual rápido	✓	✗
Mostrar progresso pra cliente	✓	✗
Edição de componente existente	✗	✓
Adicionar feature nova	✗	✓
Corrigir bug	✗	✓
Integração com API externa	✗	✓
Configurar Edge Function Supabase	✗	✓
Ajuste de copy/texto estático	✗	✓
Mudança trivial de CSS (1 cor, 1 padding)	⚠ (ok rápido)	✓
Refatoração estrutural	✗	✓

Olha essa tabela. Lovable ganha em 3 cenários. Claude Code ganha em 9. Essa é a proporção real de tempo gasto num projeto.

Se você passa 90% do tempo usando a ferramenta errada pra 90% das tarefas, claro que queima crédito. Simples assim.

Capítulo 13 — Pulo do Gato #2: Parte 2 — Executando o Fluxo

Agora o passo a passo operacional do fluxo econômico. Vou cobrir cada etapa em detalhe, incluindo os comandos exatos de terminal.

Etapa 1 — Scaffold no Lovable (1 a 3 prompts)

Já cobrimos no capítulo 6. Resumo: um prompt grande, bem estruturado, que cria o projeto inteiro de uma vez. No máximo 2-3 prompts de refinamento depois. Então: sai do Lovable.

Etapa 2 — Conectar o projeto ao GitHub

No painel do Lovable, você vai encontrar um botão de "GitHub integration" (varia de posição conforme a versão, mas sempre está presente).

1. Clique nele.
2. Autorize o Lovable a acessar seu GitHub.
3. Escolha "criar novo repositório" (ou vincular a um existente).
4. Nome do repo: geralmente o nome do seu projeto.
5. Público ou privado: pra plano grátis do Vercel, público é mais simples. Pra projetos reais, privado é seguro.
6. Confirma.

Em 30 segundos, o Lovable cria o repo no seu GitHub e faz o primeiro push. O código do seu projeto está agora no GitHub.

Etapa 3 — Clonar o repositório localmente

Abra o terminal no seu computador.

Se você nunca abriu terminal: não se assuste. É uma caixa preta onde você digita comandos. No Mac, procura "Terminal" no Spotlight. No Windows, procura "PowerShell". Linux você já sabe.

Comando pra clonar (substitua pela URL do seu repo):

```
git clone https://github.com/seu-usuario/seu-projeto.git
```

Entra na pasta:

```
cd seu-projeto
```

Pronto. Agora o código do seu projeto está no seu computador também.

Etapa 4 — Abrir o projeto no Claude Code

Se você nunca instalou o Claude Code, é rápido. Vai no site oficial da Anthropic, segue o guia (leva uns 5 minutos). Precisa de conta Anthropic com plano ativo.

Com ele instalado, dentro da pasta do projeto, basta rodar:

```
claude
```

Abre um chat. Você está dentro do Claude Code, ele está vendo os arquivos da sua pasta.

Primeiro prompt de teste:

```
Liste os arquivos do projeto e me faça um resumo rápido da estrutura.
```

Claude responde descrevendo o que ele vê: componentes, páginas, arquivos de configuração. Você está na cabine de comando.

Etapa 5 — Editando no Claude Code

A partir daqui, você usa o Claude Code pra tudo. Algumas formas comuns:

Adicionar uma feature:

```
Preciso adicionar uma página de /perfil onde o usuário pode:  
- Ver seu nome e email.  
- Editar seu nome.  
- Fazer upload de uma foto de perfil (usando Supabase Storage).  
- Cancelar assinatura Stripe.  
  
Antes de editar, me diga quais arquivos vai mexer e peça aprovação.
```

Corrigir bug:

```
Tem um bug: quando o usuário clica em "salvar nota", às vezes o salvamento dá erro mas a UI não mostra. Investigue a causa e proponha correção. Antes de aplicar, me explique.
```

Refatoração:

```
A lógica de chamar a API do Stripe está duplicada em 3 componentes.  
Extraia pra um hook customizado reutilizável. Mantenha o  
comportamento idêntico.
```

Claude Code sempre pede aprovação antes de editar arquivos. Você vê o diff, aprova ou ajusta.

Etapa 6 — Commit e push

Depois de cada bloco de mudanças (ou no final da sessão), você faz commit e push. Isso envia as mudanças de volta pro GitHub, e o Lovable sincroniza o preview.

Você pode pedir pro próprio Claude Code fazer isso:

```
Faça commit dessas mudanças com uma mensagem clara descrevendo  
o que foi feito, e faça push pro GitHub.
```

Claude Code executa os comandos Git no terminal. Pronto. Em 30 segundos o Lovable atualiza o preview, e você vê o resultado.

Etapa 7 — Lovable como visualizador

De agora em diante, Lovable é basicamente um navegador com preview do seu projeto. Você abre pra:

- Testar visualmente depois de edições.
- Compartilhar preview com cliente.
- Mostrar pra amigos.

Você NÃO edita mais no Lovable. Sua casa é o Claude Code.

Resumo em 3 frases

1. **Lovable começa, Claude Code continua, GitHub sincroniza.**
2. **Você só volta pro Lovable pra visualizar, nunca pra editar.**
3. **O custo cai de USD 80 pra USD 22 por projeto, e você ganha controle real do código.**

Capítulo 14 — Claude Code na prática pra não-devs

Este capítulo é pra quem nunca abriu terminal. Pra quem tem medo de código. Pra quem acha que "Git" é uma tacada no golfe.

Boa notícia: você não precisa virar programador. Você precisa aprender 6 comandos de terminal e entender 3 conceitos básicos de Git. Total de tempo de aprendizado: 30-40 minutos.

Os 3 conceitos de Git que você precisa

- 1. Repositório (repo)** Um repositório é a pasta do seu projeto, versionada. Todo arquivo do projeto mora ali. Pense numa pasta mágica que lembra todas as versões anteriores de cada arquivo.
- 2. Commit** Um commit é um "snapshot" da pasta num momento do tempo. Você salva uma versão do projeto dizendo "aqui, nesse estado". Depois pode voltar pra esse estado se der problema.
- 3. Push** Push é quando você envia seus commits locais (do seu computador) pro GitHub (na nuvem). É como um backup na nuvem, mas com todo o histórico.

É isso. Repositório = pasta com histórico. Commit = salvar uma versão. Push = mandar pra nuvem.

Os 6 comandos de terminal

Vou listar cada um com o que faz, quando usar, exemplo.

- 1. `cd [pasta]` — entrar numa pasta**

```
cd ~/Documents/meu-projeto
```

Te leva pra pasta do projeto. Você precisa estar dentro dela pra rodar os outros comandos.

- 2. `git clone [URL]` — baixar um repositório do GitHub**

```
git clone https://github.com/seu-usuario/seu-projeto.git
```

Usa isso uma vez, no começo, pra baixar o projeto do GitHub pro seu computador.

- 3. `git status` — ver o que mudou**

```
git status
```

Mostra quais arquivos você alterou desde o último commit. Útil pra saber o que você vai commitar.

4. `git add .` — marcar todos os arquivos alterados pra commit

```
git add .
```

O ponto no final significa "todos". Use antes de commit.

5. `git commit -m "mensagem"` — salvar uma versão

```
git commit -m "adiciona página de perfil"
```

A mensagem descreve o que mudou. Seja claro. Você vai agradecer depois.

6. `git push` — enviar pro GitHub

```
git push
```

Manda seus commits pro GitHub. O Lovable vai sincronizar automaticamente.

O fluxo padrão de trabalho

Depois de cada bloco de mudanças no Claude Code:

```
git status      # confere o que mudou
git add .       # marca tudo
git commit -m "o que você mudou, em português"
git push        # envia pro GitHub
```

Em 95% dos casos, é isso. Esses 4 comandos resolvem seu dia a dia inteiro.

Deixa o Claude Code rodar esses comandos por você

A parte linda: você não precisa digitar esses comandos sozinho. Claude Code faz pra você.

Depois de uma edição, simplesmente escreva:

```
Faça commit dessas mudanças e push pro GitHub. Mensagem:
"[descrição do que mudou]".
```

Claude Code executa os 4 comandos automaticamente. Você só aprova a execução.

Você literalmente nunca precisa digitar `git` no terminal se não quiser. Mas é bom saber o que está acontecendo por baixo.

Erros comuns do iniciante (e como resolver)

"git command not found" Git não está instalado. No Mac, vem com as "Command Line Tools" do Xcode. Pode instalar rodando `xcode-select --install`. No Windows, baixa em git-scm.com.

"permission denied" no push Você não está autenticado no GitHub. Configura usando GitHub CLI (`gh auth login`) ou SSH key. Claude Code pode te guiar nisso se acontecer.

"merge conflict" Duas versões do mesmo arquivo divergem. Raro acontecer se você é a única pessoa no projeto. Se acontecer, tira print e manda pro Gemini ou Claude Code — eles resolvem.

"your branch is behind" Significa que o Lovable alterou algo no GitHub que você ainda não baixou. Roda `git pull` pra baixar antes de dar push.

A frase pra se lembrar

"Eu não programo. Eu conversa com a ferramenta que programa."

Cola num papel. Te lembra sempre que sentir impostor.

Na Parte 5, fechamos o ciclo: troubleshooting de problemas comuns e o que fazer depois que seu produto está no ar.

PARTE 5 — Escalar e Profissionalizar

Seu produto está no ar. Agora vem o que ninguém conta: o que fazer depois.

Capítulo 15 — Troubleshooting dos erros mais comuns

Este capítulo é pra você consultar quando algo quebrar. Não leia de ponta a ponta — leia o erro que está acontecendo com você agora.

São 15 situações que eu sei que vão acontecer, porque acontecem com 90% das pessoas que fazem a trilha. Cada uma com o diagnóstico e a solução específica.

Erro 1: A IA alucinou e inventou uma função que não existe

Sintoma: Claude Code ou Lovable gerou código chamando um método tipo `supabase.auth.magicalSignIn()` e isso não existe na documentação.

Causa: Modelos generativos às vezes inventam APIs plausíveis.

Solução:

1. Abra o Gemini. Mande print do código gerado.
2. Prompt: "Esse código usa o método X. Verifique se esse método existe na documentação oficial da biblioteca. Se não existir, sugira o método correto."
3. Gemini consulta a doc e te dá o método real.
4. Volte pro Claude Code e peça: "substitua X por Y conforme a doc oficial da biblioteca".

Erro 2: Sincronização Lovable ↔ GitHub quebrou

Sintoma: Você editou no Claude Code e deu push, mas o preview do Lovable não atualizou.

Causa: Lovable pode ter dessincronizado, ou você editou direto no Lovable depois que saiu dele.

Solução:

1. No painel do Lovable, procure um botão "Sync with GitHub" ou similar.
2. Force a sincronização manual.
3. Se não resolver: faça um commit vazio pra trigger nova sincronização:

```
git commit --allow-empty -m "trigger lovable sync"
git push
```

4. Última opção: reconecte o GitHub no Lovable (desconecta e conecta de novo).

Erro 3: Supabase não conecta ("Failed to fetch")

Sintoma: O app carrega, mas qualquer ação que precisa do Supabase falha.

Causa (90% das vezes): variáveis de ambiente erradas ou ausentes.

Solução:

1. Abra o console do navegador (F12 → Console).
2. Digite: `console.log(import.meta.env)` e aperte Enter.
3. Confirme que `VITE_SUPABASE_URL` e `VITE_SUPABASE_ANON_KEY` aparecem com valores.
4. Se não aparecem: as variáveis não foram configuradas no Vercel (ou Lovable).
5. Vá em Settings → Environment Variables e adicione. Redeploy.

Erro 4: "CORS error" no console

Sintoma: Erro vermelho no console falando em CORS quando tenta acessar API externa.

Causa: A API que você está chamando não permite requisição do seu domínio.

Solução: NUNCA chame APIs de terceiros direto do frontend com chaves secretas. Crie uma Edge Function no Supabase que faz a chamada (o backend não tem problema de CORS) e o frontend chama a Edge Function.

Prompt pro Claude Code:

```
Essa chamada à API X está dando CORS error no frontend.
Mova a chamada pra uma Edge Function do Supabase e faça o
frontend chamar a Edge Function em vez da API direto.
```

Erro 5: RLS bloqueando queries que deveriam funcionar

Sintoma: Usuário logado tenta ler dados próprios e recebe array vazio. Ou inserir dá erro.

Causa: Policy de RLS mal configurada.

Solução:

1. No Supabase, vá na tabela afetada → Policies.
2. Confirme que existe policy pra SELECT (ou INSERT, conforme o caso).
3. A policy deve usar `auth.uid()` pra comparar com `user_id`.
4. Exemplo correto:

```
create policy "users read own data"
on items for select
using (auth.uid() = user_id);
```

5. Se a policy existe mas não funciona, teste no SQL Editor rodando a query diretamente — o erro retornado é mais claro.

Erro 6: Webhook do Stripe não dispara

Sintoma: Usuário pagou no Stripe, mas a assinatura não apareceu no Supabase.

Causa: várias possíveis. Vamos por ordem de probabilidade:

Solução:

1. **Checa no painel Stripe** (Events tab): o evento foi entregue? Qual status?
2. Se status é "Failed" e retorna 500: há bug na sua Edge Function. Vá nos logs.
3. Se status é "Failed" e retorna 401: a validação de `stripe-signature` está falhando. Confira o `STRIPE_WEBHOOK_SECRET` na sua Edge Function.
4. Se status é "Succeeded" mas o Supabase não atualizou: a Edge Function rodou mas tem bug na lógica de atualização. Confira os logs da Edge Function no painel Supabase.

Erro 7: Build falha no Vercel

Sintoma: Você faz push, Vercel tenta deploy, aparece "Build Failed".

Causa: várias, sempre no log.

Solução:

1. Clique no deploy falho no painel Vercel.

2. Copie o log de erro completo.
3. Cole no Gemini: "Meu build falhou no Vercel com esse log. Explique em português o que aconteceu e como corrigir." Anexe o log.
4. Gemini interpreta e te diz o que fazer.

Erros comuns encontrados aqui:

- Variável de ambiente faltando.
- Erro de TypeScript (tipo esperado X, recebeu Y).
- Import de pacote que não está no package.json.
- Case-sensitivity (nome de arquivo no Mac funciona, no Linux do Vercel não).

Erro 8: "Your branch is behind by X commits"

Sintoma: Tenta fazer `git push` e aparece esse erro.

Causa: Alguém (ou o Lovable) alterou no GitHub e você não baixou ainda.

Solução:

```
git pull
# depois
git push
```

Se aparecer conflito de merge:

1. Abra os arquivos que o terminal marcou.
2. Decida manualmente qual versão fica.
3. Ou melhor: mande pro Claude Code resolver: "tem conflito de merge nesses arquivos, resolva mantendo [a versão X]".

Erro 9: Domínio não resolve ("Site can't be reached")

Sintoma: Você apontou DNS mas o domínio não abre o site.

Causa: 3 possibilidades.

Solução:

1. **DNS ainda propagando.** Espere até 24h. Teste em <https://dnschecker.org> — se aparecer em várias regiões, já propagou.

2. **Registro errado.** Volte no Vercel → Domains e confira EXATAMENTE os valores que eles pediram. Compare com o que está no Registro.br.
3. **Conflito com outros registros.** Se você tinha o domínio apontando pra outro lugar antes, remova os registros antigos conflitantes.

Erro 10: Email vai pra spam

Sintoma: Você testa o email de boas-vindas, chega no spam (ou nem chega).

Causa: Domínio não verificado, DKIM/SPF não configurado, reputação do domínio ruim.

Solução:

1. No Resend, confirme que seu domínio está verificado (status "Verified").
2. Se não: siga o guia do Resend pra adicionar os registros DKIM/SPF no DNS do seu domínio.
3. Nunca envie de `@gmail.com` ou similares — use `@seudominio.com`.
4. Se o domínio é novo (< 1 mês), aqueça gradualmente: não mande 500 emails no primeiro dia.

Erro 11: Login com Google não redireciona de volta

Sintoma: Usuário clica em "Login com Google", autoriza, e fica numa página de erro ou em branco.

Causa (99% das vezes): redirect URI mal configurado.

Solução:

1. Google Cloud Console → Credentials → seu OAuth client.
2. Em "Authorized redirect URIs", tenha ESSAS três:

```
https://<seu-projeto>.supabase.co/auth/v1/callback
http://localhost:3000      (ou sua porta de dev)
https://seusite.com.br    (produção)
```

3. Salvar. Pode demorar até 5 min pra valer.
4. Teste de novo em aba anônima.

Erro 12: Gemini/Claude/GPT para de responder no meio

Sintoma: Você manda prompt, ele começa a responder, e trunca no meio.

Causa: limite de tokens da conversa atingido, ou erro temporário do serviço.

Solução:

1. Abra conversa nova. Copie o que precisa de contexto. Recomece com o essencial.
2. Se estiver trabalhando em algo longo no Claude, use **Projects** — contexto persiste entre conversas.
3. Evite colar 5000 linhas de código num prompt — pedaços menores são mais eficientes.

Erro 13: Supabase Storage não aceita upload

Sintoma: Você tenta fazer upload de imagem e dá erro.

Causa: bucket não existe, ou RLS do Storage não permite.

Solução:

1. Confirme que o bucket existe (painel Supabase → Storage).
2. Se o bucket é privado, confirme que as policies permitem upload pro user autenticado:

```
create policy "authenticated users upload"
on storage.objects for insert
to authenticated
with check (bucket_id = 'avatars');
```

3. No código, confirme que está usando o client autenticado (não o service_role no frontend).

Erro 14: "Crédito esgotado" no Lovable no meio do projeto

Sintoma: Você queimou o crédito do mês antes de terminar.

Causa: você não seguiu o Pulo do Gato #2.

Solução (imediata):

1. Pare de editar no Lovable AGORA.
2. Conecte ao GitHub se ainda não conectou.
3. Passa tudo pro Claude Code.
4. Espere o próximo ciclo de crédito pra usar Lovable só como preview.

Solução (definitiva): relê o Capítulo 12 e segue à risca.

Erro 15: A assinatura não libera acesso premium

Sintoma: Usuário pagou, webhook atualizou tabela, mas a UI continua tratando ele como free.

Causa: lógica de verificação de status não está lendo corretamente.

Solução:

1. No frontend, adicione log temporário: `console.log('subscription:', subscription)` .
 2. Reload. Olhe no console: o objeto subscription veio?
 3. Se veio null: a query não está rodando ou está filtrando errado.
 4. Se veio com status 'active': o bloqueio está na lógica de permissão, revise a condicional.
 5. Pro Claude Code: "quando subscription.status é 'active' ou 'trialing', liberar acesso premium. Revise o componente X."
-

Capítulo 16 — Depois de publicar: o que fazer

Você chegou no dia 30. Seu produto está no ar. Primeira venda aconteceu. E agora?

Este capítulo é curto mas importante. Porque o pior que pode acontecer é você tratar o lançamento como fim, e não como começo.

Os primeiros 7 dias pós-lançamento

Não adicione features novas. Não redesenhe nada. Não brinque com o código.

Faça apenas 3 coisas:

1. Converse com cada cliente novo pessoalmente. Manda email ou WhatsApp pra cada um dos primeiros 10-20 clientes pagantes. Pergunte:

- Como você chegou até aqui?
- O que te convenceu a pagar?
- Tem alguma parte confusa do produto?
- Qual a próxima coisa mais urgente que você precisa?

Essas conversas valem mais do que qualquer pesquisa de mercado. Eu garanto.

2. Monitore ativamente. Deixe abertos em abas:

- Dashboard do Stripe (Events tab).
- Dashboard do Supabase (Logs).
- Dashboard do Vercel (Functions + Analytics).

- Email da conta que recebe alertas.

Qualquer erro em produção, você vê em segundos e pode reagir rápido.

3. Documente o que aprendeu. Num arquivo `.md` no seu projeto, anote:

- Bugs que apareceram nos primeiros dias.
- Feedback recorrente.
- Métricas iniciais (conversão, churn precoce, tempo médio de uso).
- Decisões que você precisa tomar em breve.

Esse arquivo vai ser ouro em 60 dias.

Os 30 dias seguintes

Com base no feedback dos primeiros 7 dias, você identifica:

- **Fricção número 1** que está afastando cliente.
- **Feature número 1** que todo mundo pede.
- **Feature número 1** que ninguém usa (e que você pode remover).

Foque TODO o seu tempo nesses 3 pontos. Nada mais.

Não adicione features inventadas. Não reescreva o site porque você não gostou mais. Não mude o preço 3 vezes. Foco é o que separa o produto que sobrevive do produto que morre.

Quando buscar crescimento agressivo

A hora de apertar o acelerador de marketing é depois que você tem:

- **Pelo menos 20 clientes pagantes.**
- **Churn mensal abaixo de 15%.**
- **NPS acima de 30.**
- **Você mesmo recomendaria pro seu melhor amigo.**

Antes disso, gastar em tráfego é jogar dinheiro num balde furado. Consertos de produto vêm primeiro.

Quando chegar a hora, o Pulo do Gato #1 + #2 continuam te servindo. Você vai precisar de:

- Landing com variações A/B.
- Criativos de ad em volume.

- Sequências de email mais sofisticadas.
- SEO de verdade.

Tudo isso é GPT e Manus delegando trabalho pesado. Você continua orquestrando.

Quando sair do stack no-code

Em algum momento você vai ouvir: "ih, isso aí não escala, você precisa migrar pra AWS/K8s/microserviços".

95% das vezes isso é bobagem. Supabase + Vercel + Stripe aguenta fluxos de milhões de reais/ano tranquilamente. Empresas listadas usam essa stack.

Só considere migrar se:

- Custo mensal de Supabase passa de USD 500 (aí é hora de conversar com eles pra plano customizado).
- Você tem caso de uso bem específico (ex: ML pesado, processamento de vídeo).
- Você contratou time de 3+ devs experientes.

Antes disso: fica onde está. Resolve o problema do cliente. Fatura.

Virando autoridade no seu nicho

Algo que acontece depois que o produto está rodando: você percebe que sabe MUITO sobre o nicho. Mais do que a maioria. E começa a ser procurado:

- "Como você fez?"
- "Consegue construir algo parecido pra mim?"
- "Dá uma consultoria?"

Três caminhos se abrem:

1. Continuar focado no produto. Ignora as solicitações. Fatura mais.

2. Fazer consultoria de alto ticket. Cobrar R\$ 5-15k por projeto entregue a clientes. Exige tempo, mas margem altíssima.

3. Ensinar o método. Se você se interessar por ensinar, crie seu próprio curso, ebook, mentoria. O mercado brasileiro em 2026 paga bem por esse tipo de conhecimento prático.

Escolha um. Os três dão certo. O segredo é não tentar fazer os três ao mesmo tempo.

Encerramento do ebook

Você chegou até aqui. Parabéns.

Se você está lendo isso sem ter começado ainda, entendi. Feche o PDF. Abre o Dashboard interativo. Começa a Fase 1. Volta aqui quando travar.

Se você está lendo isso tendo feito alguma parte da trilha, você já sabe algo que 99% das pessoas não vão saber nunca: **método vence talento, consistência vence intensidade, e entregar imperfeito vence não entregar.**

A IA não é mágica. Ela é alavanca. Sem ponto de apoio (seu cérebro, sua clareza, seu critério), alavanca não levanta nada.

Este ebook foi o ponto de apoio. A alavanca agora é sua.

Construa coisas. Publique mesmo imperfeitas. Conversa com clientes de verdade. Ajusta. Repete.

Em 12 meses, você vai ler sua primeira versão deste ebook e achar que era só o começo.

Boa obra.

APÊNDICES

Apêndice A — Glossário técnico sem vergonha

Termos que aparecem o tempo todo. Cada um em 1-2 frases, sem jargão.

API Um jeito padronizado de um programa pedir informação ou ação pra outro programa. Tipo um cardápio: você escolhe o prato (endpoint), passa detalhes (parâmetros), e recebe o prato (resposta).

Auth (autenticação) Processo de provar quem você é no sistema. Login e senha, Google OAuth, etc. Diferente de autorização (que define o que você pode fazer depois de logado).

Backend A parte do sistema que roda no servidor, longe do usuário. Onde ficam os dados seguros, as regras de negócio, as chaves secretas. Oposto de frontend.

Bucket (Storage) Uma "pasta" no Supabase Storage onde você guarda arquivos (imagens, PDFs). Cada bucket tem suas regras de quem pode ler/escrever.

Build O processo de transformar o código fonte em arquivos otimizados prontos pra rodar em produção. Onde erros aparecem antes do usuário ver.

CORS Uma regra de segurança do navegador que impede um site de chamar APIs de outros domínios sem permissão. Causa o erro mais comum pra iniciante.

CRUD Sigla de Create, Read, Update, Delete — as 4 operações básicas em dados. Todo SaaS é CRUD + regras de negócio.

DNS Sistema que traduz nome de domínio (seudominio.com) pra endereço IP (192.168.1.1). Quando você compra domínio, precisa configurar DNS pra apontar pro seu site.

Edge Function Código backend que roda perto do usuário (em servidor "na borda" da rede). Perfeito pra webhooks e APIs rápidas.

Endpoint Uma URL específica de uma API que faz uma coisa específica. Tipo `/api/users` pra listar usuários.

Environment Variables Chaves e configurações que mudam entre ambientes (dev, prod). Você não coloca no código — coloca em variáveis que o servidor lê.

Frontend A parte do sistema que o usuário vê no navegador. HTML, CSS, JS, React. Oposto de backend.

Hook (React) Função especial do React que te dá acesso a capacidades (estado, efeitos, contexto). `useState`, `useEffect`, etc. Você não precisa entender a fundo — só usar.

HTTPS Versão segura do HTTP (os dados trafegam criptografados). Todo site profissional usa HTTPS. Vercel faz automático.

JSON Formato de dados estruturados usado em APIs. Parece objeto JavaScript. Tipo: `{"nome": "João", "idade": 30}`.

OAuth Protocolo que permite "Login com Google/Facebook/etc". Você autoriza o outro sistema a confirmar sua identidade.

ORM Ferramenta que faz queries no banco usando código em vez de SQL direto. Supabase cliente é uma espécie de ORM.

PRD (Product Requirements Document) Documento que descreve o que o produto faz, pra quem, por quê, como. Primeira entrega antes de qualquer código.

Push (Git) Enviar seus commits locais pro repositório remoto (GitHub).

React Biblioteca de JavaScript pra construir interfaces. Base do Lovable, do Next.js, de quase tudo moderno.

Repository (repo) Uma pasta versionada com histórico. Normalmente hospedada no GitHub.

RLS (Row Level Security) Recurso do PostgreSQL (e Supabase) que filtra automaticamente quais linhas cada usuário pode ver/editar. Obrigatório pra segurança em SaaS multi-tenant.

Schema Estrutura do banco de dados — quais tabelas existem, quais colunas, quais tipos, quais relações.

SSR (Server-Side Rendering) Gerar o HTML no servidor antes de mandar pro navegador. Melhor pra SEO e performance inicial.

SSL/TLS Protocolos de criptografia que tornam o HTTPS seguro. Vercel gerencia automaticamente.

TypeScript JavaScript com tipos. Ajuda a pegar bugs antes do usuário ver. Todo SaaS profissional usa.

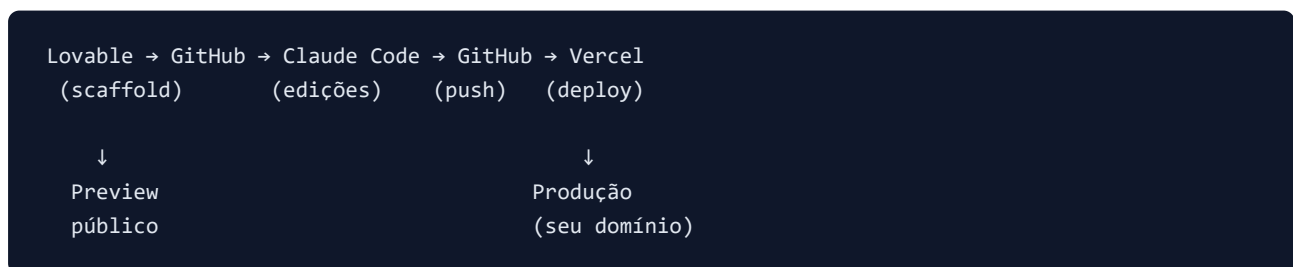
Webhook Um endpoint que RECEBE notificações de outros sistemas. Tipo: Stripe te manda um webhook quando alguém pagou.

Apêndice B — Stack Map Visual

Referência rápida: qual ferramenta faz o quê no seu projeto.



Fluxo de desenvolvimento:



Fluxo de dinheiro:

```
Cliente → Stripe Checkout → Stripe → Webhook → Supabase
      (cartão ou Pix)          (Edge Fn) (subscriptions)
                              |
                              ▼
                        Frontend libera
                        acesso premium
```

Apêndice C — Cheat sheet Git pra não-devs

Fluxo padrão diário

```
cd caminho/do/projeto    # entrar na pasta
git pull                 # baixar alterações
# ... edita com Claude Code ...
git add .                # marcar tudo
git commit -m "mensagem" # salvar versão
git push                 # enviar pro GitHub
```

Consultas

```
git status               # o que mudou
git log --oneline -10    # últimos 10 commits
git diff                  # diferenças desde último commit
```

Desfazer coisas

```
git checkout arquivo     # restaurar arquivo
git reset --soft HEAD~1  # desfazer último commit, manter mudanças
git reset --hard HEAD~1  # desfazer último commit, descartar (cuidado!)
```

Situações especiais

```
git pull --rebase        # sincronizar mantendo linha do tempo linear
git stash                # guardar mudanças sem commit
git stash pop             # trazer de volta
```

Apêndice D — Cheat sheet de comandos Claude Code

Dentro da sessão do Claude Code, você pode usar esses prompts-padrão:

Exploração

Liste os arquivos do projeto e me faça um resumo da estrutura.

Leia o arquivo X e me explique o que ele faz em 3 bullets.

Implementação

Adicione a feature [X]. Antes de editar, liste os arquivos que vai mexer e peça aprovação.

Debug

Tem um bug: [descreva sintoma]. Investigue a causa e proponha correção.

Refatoração

A lógica X está duplicada em vários lugares. Extraia pra um [hook/função/módulo] reutilizável. Mantenha o comportamento idêntico.

Auditoria

Audite o arquivo X quanto a:

1. Bugs
2. Segurança (chaves, validação, RLS)
3. Performance
4. Manutenibilidade

Ordene do mais crítico ao menor.

Git

Faça commit dessas mudanças com uma mensagem clara descrevendo o que foi alterado. Depois push pro GitHub.

Perguntas

Por que você fez X dessa forma? Explique a decisão.

Tem uma forma mais simples de fazer isso?

Apêndice E — Sinais de que você está fazendo certo (e de que está fazendo errado)

Você está fazendo CERTO se:

- Tem PRD antes de ter código.
- Faz commit várias vezes por dia com mensagens claras.
- Consulta este ebook quando trava (em vez de tentar na raça).
- Conversa com potenciais clientes antes e depois de publicar.
- Mede coisas (cadastros, conversão, uso).
- Decide feature com base em dor real, não em "acho legal".
- Reserva tempo pra auditoria antes de cada deploy.

Você está fazendo ERRADO se:

- Pula o PRD achando que "ideia é simples".
 - Fica 2 horas tentando "descobrir sozinho" em vez de perguntar pra IA certa.
 - Edita tudo no Lovable e queima crédito.
 - Reescreve o projeto inteiro porque "não gostou mais".
 - Adiciona features sem consultar cliente.
 - Não tem Stripe em test mode antes de live.
 - Posta landing sem auditoria nem testes responsivos.
 - Publica sem RLS configurado.
-